

ЧЕРНІВЕЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ЮРІЯ ФЕДЬКОВИЧА

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ
КОМП'ЮТЕРНИХ СИСТЕМ

**КУРСОВИЙ ПРОЄКТ:
СТРУКТУРИ ДАНИХ, АЛГОРИТМИ
ТА ОБ'ЄКТНО-ОРІЄНТОВАНА ПАРАДИГМА**

на тему: Розробка системи для управління
касою аерофлоту

здобувач вищої освіти II курсу 243 групи
спеціальності 121 «Інженерія програмного
забезпечення»
першого (бакалаврського) рівня вищої освіти
Граневський Б. Я.

Оцінка:

за національною шкалою

(словами)

кількість балів

(цифра)

за шкалою ECTS

(літера)

Чернівці, 2025

ЧЕРНІВЕЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ЮРІЯ ФЕДЬКОВИЧА

Навчально-науковий інститут фізико-технічних та комп'ютерних наук
Кафедра програмного забезпечення комп'ютерних систем

ЗАВДАННЯ
на курсовий проєкт
здобувачу вищої освіти
першого (бакалаврського) рівня вищої освіти
Граневському Богдану Ярославовичу

- 1) Тема проєкту: Розробка системи автоматизації відділу кадрів.
- 2) Вхідні дані до проєкту:
 - опис предметної області - структура даних про авіарейси, користувачів та квитки;
 - початкові набори тестових даних про рейси та користувачів;
 - методичні матеріали та вимоги до функціональності програмної системи
- 3) Вихідні дані до проєкту:
 - консольна програма для управління касою аерофлоту;
 - база даних, що формується та оновлюється під час роботи системи;
 - опис структури програмної системи та реалізованих класів;
 - результати тестування роботи;
 - матеріали для захисту;

Керівники проєкту

(підпис керівника)

Валь О.О.

(підпис керівника)

Дяконенко Б.В.

(підпис керівника)

Комісарчук В.В.

Завдання взяв до виконання

(підпис студента)

Граневський Б.Я.

РЕФЕРАТ

Курсовий проєкт налічує 73 сторінок, 32 рисунки, 7 таблиць та 7 джерел.

У межах курсового проєкту створено консольний застосунок для обліку авіарейсів та роботи з даними про польоти. Програма функціонує на комп'ютерах під керуванням ОС Windows і дає змогу користувачеві виконувати основні операції з базою даних рейсів CSV файлах.

Система зберігає відомості про рейси – номер рейсу, маршрут, проміжні пункти, час відправлення, тривалість перельоту та кількість доступних місць. Реалізовано можливість додавання нових рейсів, зміни вже існуючих записів, видалення застарілої інформації, пошуку за різними критеріями, сортування й фільтрації списку рейсів. Окремо передбачено роботу з обліковими записами користувачів, розмежування прав доступу та оформлення бронювання місць із формуванням посадкових талонів.

Розробка виконана мовою програмування C++ із використанням текстових файлів формату CSV для зберігання даних про рейси та окремого файлу формату TXT для обліку користувачів. Архітектура програми побудована з використанням об'єктно-орієнтованих підходів, із виділенням окремих класів для управління рейсами, користувачами та файловими операціями. Така структура полегшує внесення змін і подальший розвиток системи.

СИСТЕМА БРОНЮВАННЯ, АВІАРЕЙСИ, C++, ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ, CSV, АЛГОРИТМИ ПОШУКУ, КОНСОЛЬНИЙ ІНТЕРФЕЙС.

SUMMARY

The course project contains 73 pages, 32 figures, 7 tables, and 7 references.

The present course project is devoted to the development of a console application for managing a simple flight database. The program is intended to run on personal computers under the Windows operating system and provides basic tools for working with flight information stored in CSV files.

The system keeps records of flights including the flight number, route, intermediate stops, departure time, flight duration and number of available seats. The application supports adding new flights, updating and deleting existing records, searching by different parameters, sorting and filtering the list of flights. Separate functionality is implemented for user account management, access control (administrator / regular user) and booking seats for a selected flight with issuing a text-based boarding pass.

The project is implemented in C++ and uses CSV text files to store flight information together with an additional file for user data. The architecture follows object-oriented principles and is divided into several modules that are responsible for flight management, user management and file operations. This modular structure makes the application easier to maintain and extend.

BOOKING SYSTEM, AIR FLIGHTS, C++, OBJECT-ORIENTED PROGRAMMING, CSV, SEARCH ALGORITHMS, CONSOLE INTERFACE.

ЗМІСТ

ЗМІСТ	5
ПЕРЕЛІК СКОРОЧЕНЬ ТА УМОВНИХ ПОЗНАЧЕНЬ.....	6
РОЗДІЛ 1. СПЕЦИФІКАЦІЯ ПРОЄКТУ	7
1.1. ПОСТАНОВКА ЗАВДАННЯ	7
1.2. ОПИС ВИМОГ ДО ПРОГРАМНОЇ СИСТЕМИ	9
РОЗДІЛ 2. ПРОГРАМНА ДОКУМЕНТАЦІЯ	18
2.1. АЛГОРИТМ РОЗВ’ЯЗАННЯ ЗАДАЧІ	18
2.2. ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ.....	23
2.3. ОПИС ПРОГРАМНОЇ СИСТЕМИ	28
2.4. ТЕСТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ	37
2.5. ПРОГРАМНІ ЗАСОБИ РОЗРОБКИ	39
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	41
ДОДАТКИ	42

ПЕРЕЛІК СКОРОЧЕНЬ ТА УМОВНИХ ПОЗНАЧЕНЬ

№	Скорочення / Позначення	Пояснення
1	БД	База даних
2	ООП	Об'єктно-орієнтоване програмування
3	ОС	Операційна система
4	ПЗ	Програмне забезпечення
5	CRUD	Сукупність базових операцій над даними (Create – створення, Read – читання, Update – оновлення, Delete – видалення)
6	CSV	Comma-Separated Values - текстовий формат подання табличних даних, де значення полів розділяються комами
7	ID	Identifier - унікальний ідентифікатор, що використовується для розпізнавання об'єкта в системі
9	UI	User Interface - інтерфейс користувача
10	STL	Standard Template Library - стандартна бібліотека шаблонів C++, що містить контейнери list, vector тощо
11	TXT	Формат текстового файлу, що використовується для збереження неструктурованих або слабоструктурованих даних
12	GUI	Graphical User Interface – графічний інтерфейс користувача

РОЗДІЛ 1. СПЕЦИФІКАЦІЯ ПРОЄКТУ

1.1. Постановка завдання

Предметною областю є інформаційна система для роботи з базою даних авіарейсів, яка охоплює процеси додавання, перегляду, редагування, видалення та пошуку інформації про рейси, а також бронювання місць пасажиром.

Система повинна підтримувати зберігання основних відомостей про рейси, а саме – номер рейсу, пункт відправлення та пункт призначення, проміжні зупинки, час відправлення, тривалість перельоту, загальну кількість місць та кількість вільних місць.

Доступ до основних функцій, що пов'язані з маніпуляцією даними надається тільки після успішної авторизації. Для цього використовується файл у якому зберігається інформація про облікові записи користувачів.

Система повинна забезпечувати:

- ідентифікацію користувачів під різними ролями (адміністратор / користувач);
- ведення єдиної бази даних рейсів у текстовому файлі формату CSV;
- перегляд поточної інформації про всі доступні рейси;
- додавання нових рейсів із повним набором необхідних атрибутів;
- редагування даних існуючих рейсів (час відправлення, кількість місць і тд);
- видалення рейсів, які більше не виконуються або введені помилково;
- пошук рейсів за номером, напрямком або іншими ключовими атрибутами;
- сортування та фільтрацію списку рейсів за обраними критеріями;
- оформлення бронювання місць для користувача і формування текстового посадкового талона.

Розроблювана система повинна бути стійкою до помилкових дій користувача, зберігати цілісність даних та забезпечувати коректний запис інформації у файли при виконанні всіх операцій.

1.2. Опис вимог до програмної системи

Вимоги до інтерфейсу користувача

- 1) Робоча мова інтерфейсу – українська.
- 2) Стиль повідомлень – формулювання мають бути зрозумілими користувачеві без спеціальної термінології.
- 3) Меню навігації – інтерфейс будується у вигляді текстового меню, яке забезпечує:
 - авторизацію та реєстрацію користувачів;
 - роботу з рейсами;
 - оформлення та перегляд бронювання;
 - вихід із програми.
- 4) Підтримка інструкцій – у програмі має бути присутній пункт меню який виводить коротку інструкцію для користувача з описом основних можливостей системи та способу навігації по меню.
- 5) Обробка помилок введення – у випадку введення некоректних значень програма повинна виводити повідомлення про помилку й пропонувати повторити введення.

Вимоги до архітектури програми

- 1) Мова реалізації – C++
- 2) Зберігання даних – звичайні текстові файли:
 - рейси – у файлі формату CSV;
 - користувачі – в окремому текстовому файлі з логіном, паролем і роллю;
- 3) Структура коду – для основних сутностей є окремі класи:
 - клас рейсу, клас користувача;
 - менеджери які відповідають за роботу з колекціями рейсів і користувачів;
 - допоміжний клас для роботи з файлами;
- 4) Надійність – програма не повинна втрачати своєї дієздатності від неправильного вводу. Перед важливими операціями мають бути базові перевірки й захист від очевидних помилок.

Вимоги до функціональності програми

1) Облікові записи:

- реєстрація нового користувача;
- вхід у систему за логіном і паролем;

2) Робота з рейсами:

- перегляд списку всіх рейсів;
- додавання нового рейсу;
- редагування та видалення існуючих рейсів;
- пошук, просте сортування й фільтрація рейсів за основними критеріями;

2) Бронювання:

- вибір рейсу й перевірка, чи є вільні місця;
- оформлення бронювання та зменшення кількості вільних місць;
- виведення посадкового талона в текстовому вигляді;
- перегляд своїх попередніх бронювань;

3) Сервісні можливості:

- показ короткої інструкції користувача;
- коректний вихід із програми з збереженням змін у файлах.

Опис сценаріїв використання програми

Сценарій 1: Авторизація користувача	
Передумова: Користувач знаходиться у головному меню	
Основний сценарій	
Крок 1	Користувач обирає пункт увійти як admin/user
Крок 2	Користувач вводить логін та пароль
Крок 3	Користувач входить в меню адміністратора/звичайного користувача
Альтернативний сценарій	
Крок 1	Користувач обирає пункт увійти як адмін/звичайни користувач
Крок 2	Користувач вводить логін та пароль
Крок 3	Програма не знаходить обліковий запис із таким логіном
Крок 4	Програма виводить повідомлення про помилку, користувач повертається до головного меню

Сценарій 2: Перегляд списку усіх рейсів	
Передумова: Користувач авторизований як admin/user, БД завантажена	
Основний сценарій	
Крок 1	Користувач обирає пункт «Показати список всіх рейсів»
Крок 2	Програма звертається до списку рейсів
Крок 3	Програма виводить список усіх рейсів у консоль
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Показати список всіх рейсів»
Крок 2	Програма звертається до списку рейсів
Крок 3	Програма виводить помилку, список попрожній

Сценарій 3: Пошук рейсу за критерієм	
Передумова: Користувач авторизований як admin/user, БД завантажена	
Основний сценарій	
Крок 1	Користувач обирає пункт «Пошук за критерієм»
Крок 2	Користувач обирає критерії пошуку
Крок 3	Користувач вводить значення для пошуку
Крок 4	Програма знаходить рейс із шуканими значеннями та виводить в консоль
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Пошук за одним критерієм»
Крок 2	Користувач вводить некоректні значення
Крок 3	Програма виводить помилку і повертає до меню

Сценарій 4: Сортування рейсів	
Передумова: Користувач авторизований як admin/user, БД завантажена	
Основний сценарій	
Крок 1	Користувач обирає пункт «Сортувати дані»
Крок 2	Користувач обирає параметр сортування(зростання/спадання)
Крок 3	Користувач обирає ключ сортування(id, назва, час польоту і тд)
Крок 4	Програма виводить відсортований список рейсів в консоль
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Сортувати дані»
Крок 2	Користувач вводить некоректні значення
Крок 3	Програма виводить помилку і повертає до меню

Сценарій 5: Фільтрація рейсів	
Передумова: Користувач авторизований як admin/user, БД завантажена	
Основний сценарій	
Крок 1	Користувач обирає пункт «Фільтрувати дані»
Крок 2	Користувач вводить значення для фільтрації
Крок 3	Програма виводить відфільтрований список
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Фільтрувати дані»
Крок 2	Користувач вводить неіснуючі значення
Крок 3	Програма виводить повідомлення про неможливість фільтрації і повертає до меню

Сценарій 6: Придбання квитка	
Передумова: Користувач авторизований як user, БД завантажена	
Основний сценарій	
Крок 1	Користувач обирає пункт «Придбати квиток»
Крок 2	Користувач вводить пункт призначення
Крок 3	Програма виводить найближчий по часу рейс до місця призначення
Крок 4	Користувач вводить кількість квитків, які хоче придбати
Крок 5	Програма виводить повідомлення про успішне придбання
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Придбати квиток»
Крок 2	Програма виводить найближчий по часу рейс до місця призначення
Крок 3	Користувач вводить надмірну кількість квитків, які хоче придбати
Крок 4	Програма виводить повідомлення, що така кількість квитків на обраний рейс відсутні і повертає до меню

Сценарій 7: Перегляд історії посадкових талонів	
Передумова: Користувач авторизований як user, БД завантажена	
Основний сценарій	
Крок 1	Користувач обирає пункт «Перегляд історії посадкових талонів»
Крок 2	Програма звертається до БД і шукає записи які містять логін користувача
Крок 3	Програма виводить список посадкових талонів користувача
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Перегляд історії посадкових талонів»
Крок 2	Програма звертається до БД і шукає записи які містять логін користувача
Крок 3	Програма не знаходить даних де міститься логін користувача і виводить повідомлення про помилку, повертає до меню

Сценарій 8: Зміна логіну користувача	
Передумова: Користувач авторизований як user	
Основний сценарій	
Крок 1	Користувач обирає пункт «Змінити ім'я користувача»
Крок 2	Користувач вводить новий логін
Крок 3	Програма виводить повідомлення про успішну зміну логіну
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Змінити ім'я користувача»
Крок 2	Користувач вводить існуючий/порожній логін
Крок 3	Програма виводить помилку та повертає до меню

Сценарій 9: Зміна паролю користувача	
Передумова: Користувач авторизований як user/admin	
Основний сценарій	
Крок 1	Користувач обирає пункт «Змінити пароль»
Крок 2	Користувач вводить старий та новий пароль
Крок 3	Програма виводить повідомлення про успішну зміну паролю
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Змінити пароль»
Крок 2	Користувач вводить некоректний старий пароль
Крок 3	Програма виводить помилку та повертає до меню

Сценарій 10: Перегляд інформації про профіль користувача	
Передумова: Користувач авторизований як user/admin	
Основний сценарій	
Крок 1	Користувач обирає пункт «Показати інформацію профілю»
Крок 2	Програма виводить профіль користувача
Альтернативний сценарій	

Сценарій 11: Перегляд інструкції користувача	
Передумова: Користувач авторизований як user/admin	
Основний сценарій	
Крок 1	Користувач обирає пункт «Інструкція користувача»
Крок 2	Програма виводить інструкцію користувача в консоль
Альтернативний сценарій	

Сценарій 12: Вихід із облікового запису	
Передумова: Користувач авторизований як user/admin	
Основний сценарій	
Крок 1	Користувач обирає пункт «Вийти»
Крок 2	Програма завершує робочу сесію та повертає користувача до головного меню
Альтернативний сценарій	

Сценарій 13: Додавання нового користувача	
Передумова: Користувач авторизований як admin	
Основний сценарій	
Крок 1	Користувач обирає пункт «Змінити пароль»
Крок 2	Користувач вводить логін та пароль нового користувача
Крок 3	Програма перевіряє унікальність логіну та паролю. Вони унікальні
Крок 4	Програма виводить повідомлення про успішне додавання нового користувача
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Змінити пароль»
Крок 2	Користувач вводить логін та пароль нового користувача
Крок 3	Програма перевіряє унікальність логіну та паролю. Вони не унікальні
Крок 4	Програма виводить помилку і повертає до меню

Сценарій 14: Видалення користувача	
Передумова: Користувач авторизований як admin	
Основний сценарій	
Крок 1	Користувач обирає пункт «Видалити користувача»
Крок 2	Користувач вводить логін
Крок 3	Програма виводить повідомлення про успішне видалення користувача
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Видалити користувача»
Крок 2	Користувач вводить неіснуючий/свій логін
Крок 3	Програма виводить помилку та повертає до меню

Сценарій 15: Перегляд списку користувачів	
Передумова: Користувач авторизований як admin, завантажена БД	
Основний сценарій	
Крок 1	Користувач обирає пункт «Показати список користувачів»
Крок 2	Програма виводить список користувачів
Альтернативний сценарій	

Сценарій 16: Додавання нового рейсу	
Передумова: Користувач авторизований як admin, завантажена БД	
Основний сценарій	
Крок 1	Користувач обирає пункт «Додати новий рейс»
Крок 2	Користувач вводить дані про рейс
Крок 3	Програма виводить повідомлення про успішне створення рейсу
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Додати новий рейс»
Крок 2	Користувач вводить некоректні дані про рейс
Крок 3	Програма виводить помилку та повертає до меню

Сценарій 17: Редагування рейсу	
Передумова: Користувач авторизований як admin, завантажена БД	
Основний сценарій	
Крок 1	Користувач обирає пункт «Редагувати рейс»
Крок 2	Користувач вводить номер рейсу
Крок 3	Користувач обирає бажаний параметр та редагує значення
Крок 4	Програма виводить повідомлення про успішне редагування
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Редагувати рейс»
Крок 2	Користувач вводить некоректний номер рейсу або один із параметрів
Крок 3	Програма виводить помилку та повертає до меню

Сценарій 18: Видалення рейсу	
Передумова: Користувач авторизований як admin, завантажена БД	
Основний сценарій	
Крок 1	Користувач обирає пункт «Видалити рейс»
Крок 2	Користувач вводить номер рейсу
Крок 3	Рейс знайдено. Програма виводить повідомлення про видалення рейсу
Альтернативний сценарій	
Крок 1	Користувач обирає пункт «Видалити рейс»
Крок 2	Користувач вводить некоректний номер рейсу
Крок 3	Програма виводить повідомлення про впомилку та повертає до меню

РОЗДІЛ 2. ПРОГРАМНА ДОКУМЕНТАЦІЯ

2.1. Алгоритм розв'язання задачі

Узагальнені блок-схеми алгоритмів додавання об'єкту, редагування об'єкту та видалення об'єкта наведено на рис.1-3.

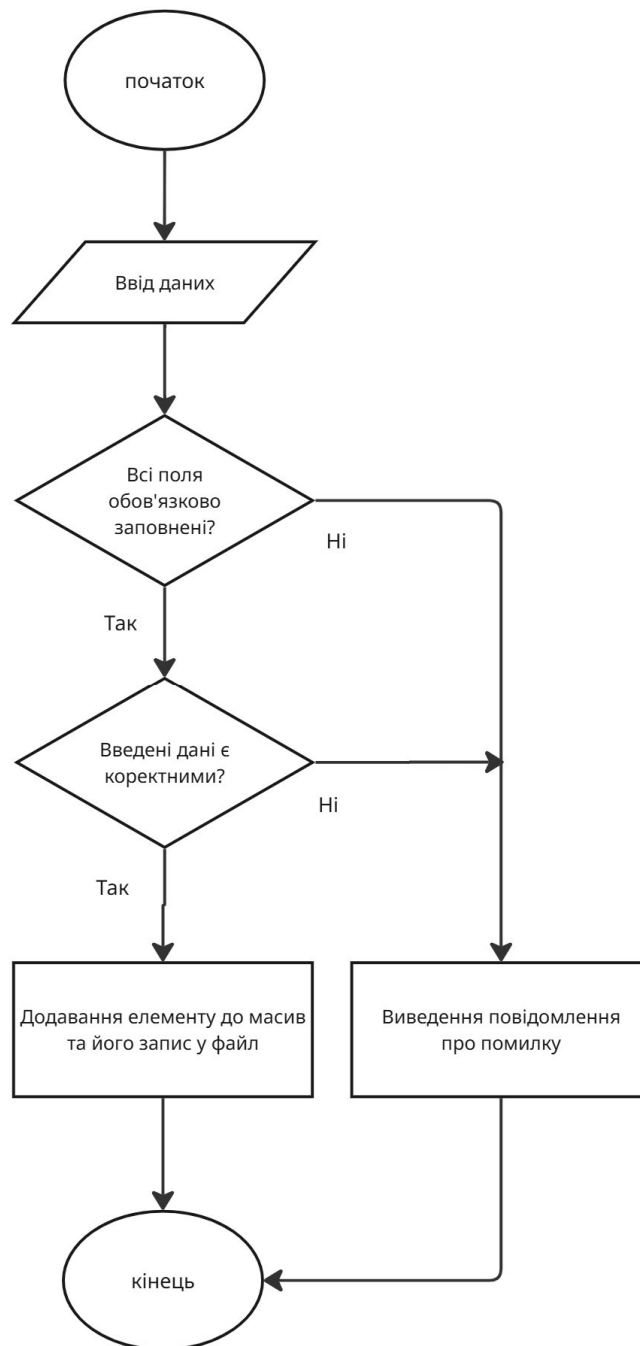


Рисунок 1 – Блок-схема алгоритму додавання об'єкту

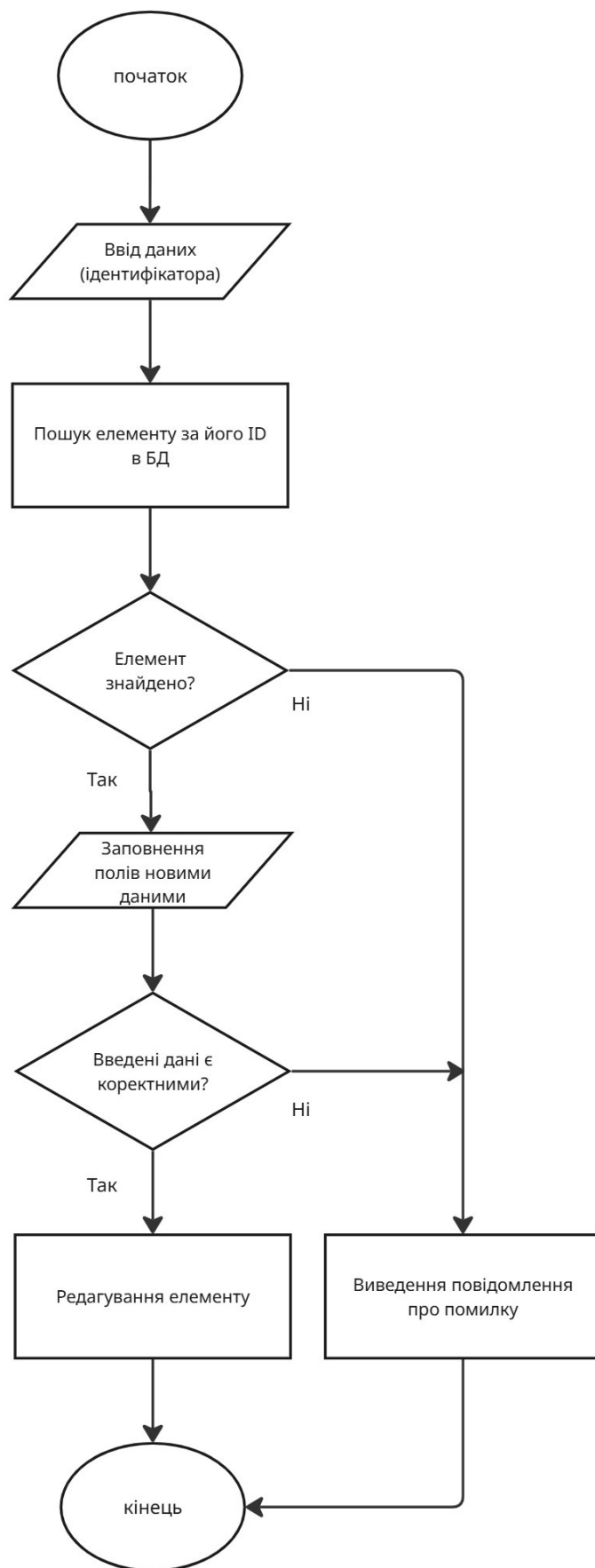


Рисунок 2 – Блок-схема алгоритму редагування об'єкту

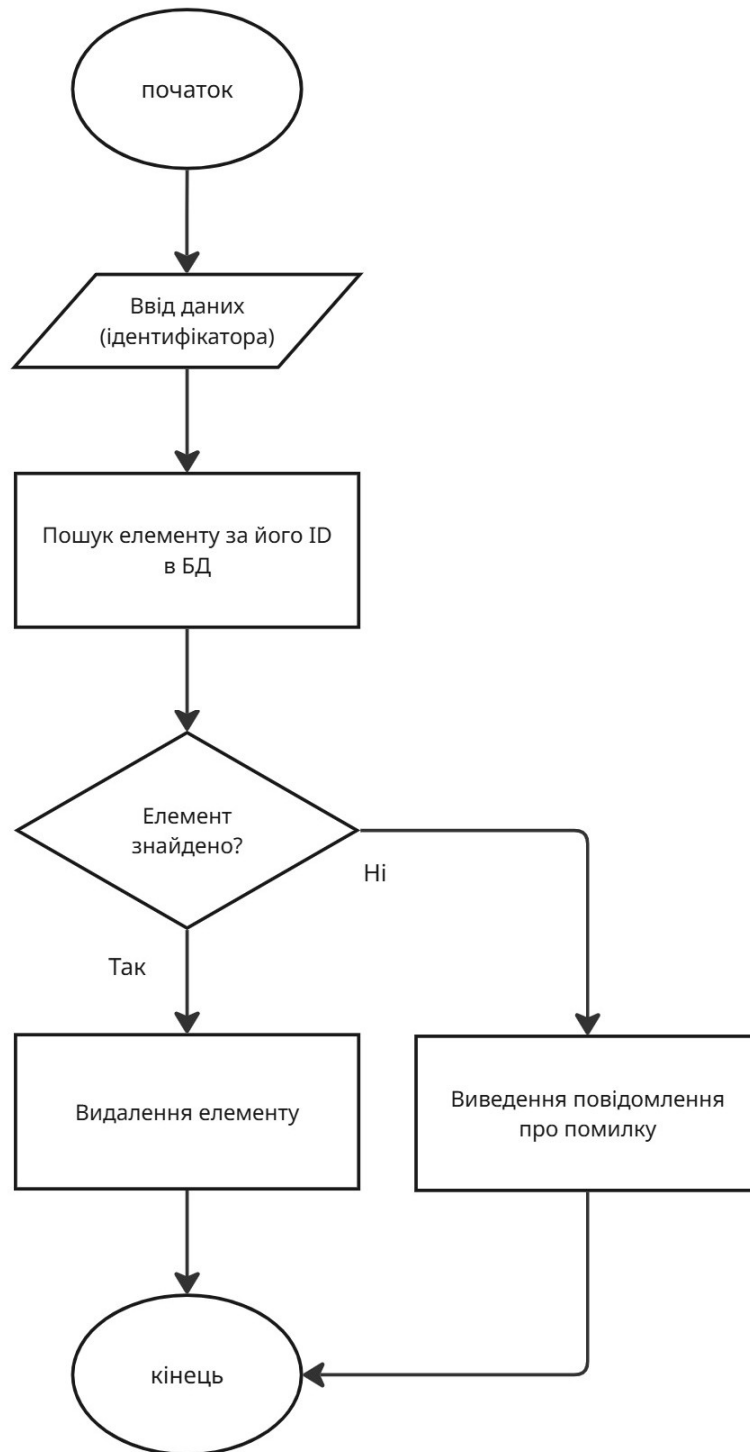


Рисунок 3 – Блок-схема алгоритму видалення об'єкта

Узагальнені блок-схеми алгоритмів пошуку об'єкта, перегляду доданих об'єктів та їх сортування наведено на рис.4-5.

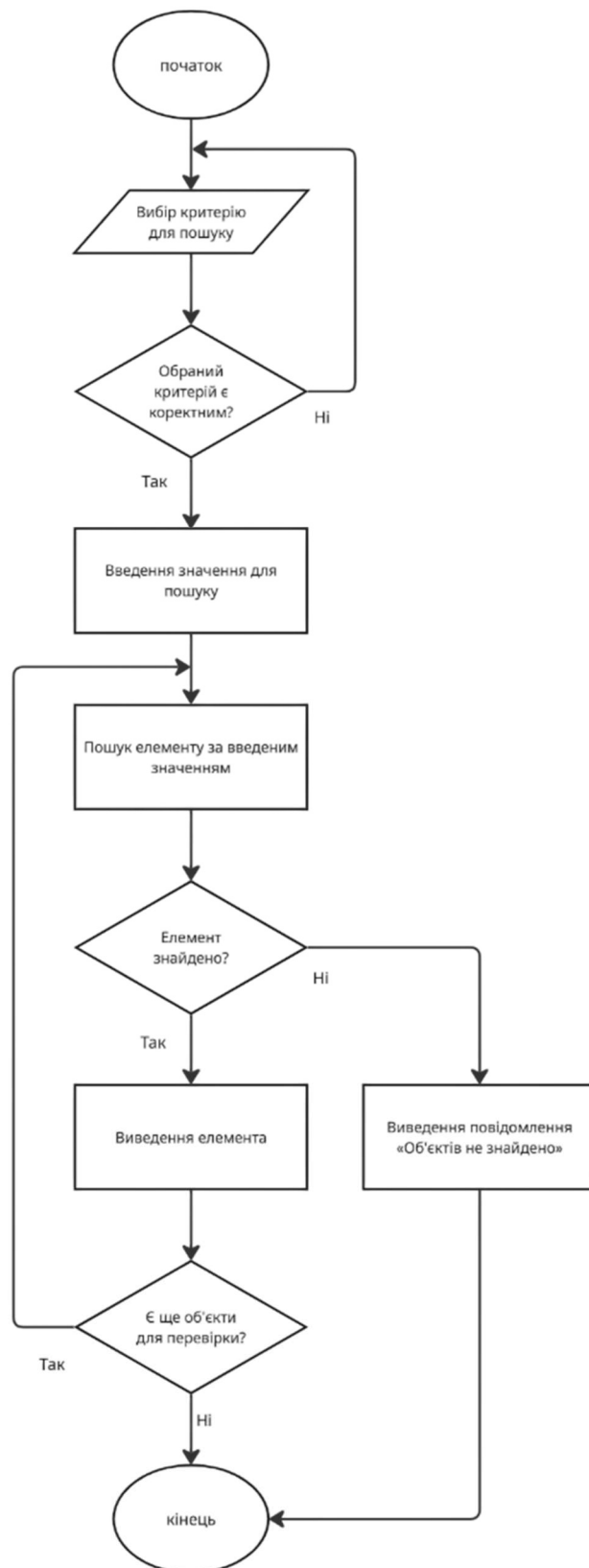


Рисунок 4 – Узагальнена блок-схема алгоритму пошуку та виведення об'єкта

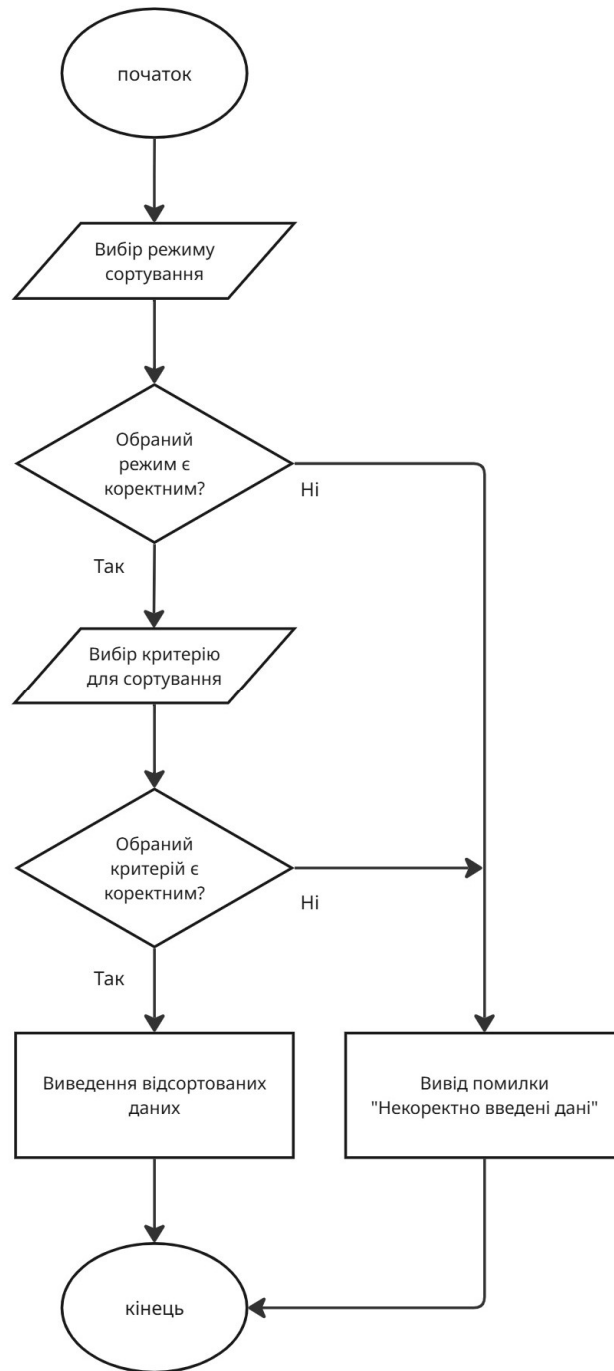


Рисунок 5 – Блок-схема алгоритму сортування та виводу елементів

2.2. Проектування програмної системи

Робота розробленого програмного забезпечення реалізується наступними програмними класами:

- 1) IManagable.
- 2) FlightManager.
- 3) UserManager.
- 4) Flight.
- 5) User.
- 6) FileHandler

В табл.1-6 подано документування класів.

Таблиця 1 – Документування класу *IManagable*

Характеристика		Опис
Назва класу		IManagable
Опис класу		Інтерфейсний клас, який задає базовий набір методів для класів менеджерів
Залежності від інших класів		—
Опис атрибутів (полів)		—
Опис методів		
1	add()	Абстрактний метод, що виводить всю інформацію про об'єкт
2	remove()	Абстрактний метод, що виводить коротку інформацію про об'єкт
3	edit()	Абстрактний метод що повертає зарплату об'єкта-робітника
4	search()	Абстрактний метод що повертає булеве значення щодо зміни статусу об'єкта-робітника
5	sortData()	Абстрактний метод що повертає булеве значення щодо можливості зміни статусу об'єкта-робітника
6	filter()	Абстрактний метод, що записує дані об'єкта в файл

Таблиця 2 – Документування класу *FlightManager*

Характеристика		Опис
Назва класу		FlightManager
Опис класу		Клас нащадок, клас менеджер для керування колекцією об'єктів Flight. Спрямований на завантаження, зберігання, додавання, редагування, видалення, пошук, сортування, фільтрацію та бронювання рейсів
Залежності від інших класів		IManagable – клас інтерфейс – та FileHandler – абстрактний клас
Опис атрибутів (полів)		flights(список усіх рейсів завантажених із файлу “flights.csv”)
Опис методів		
1	loadFromFile()	Зчитує всі рейси із CSV файлу “flights.csv” та зберігає їх у список flights
2	saveToFile()	Записує актуальний список рейсів flights у CSV файл “flights.csv”.
3	showFlightList(list<Flight> list)	Виводить на екран переданий список рейсів
4	findNearestFlight(string destination)	Шукає найближчий за часом рейс до пункту призначення
5	bookTickets(User& user)	Ініціює процес оформлення бронювання для обраного рейсу
6	generateBoardingPass(User& user, Flight& flight, int ticketCount)	Створює посадкову відомість і записує їх у файл “BoardingPasses.txt”
7	getFlights()	Повертає поточний список усіх рейсів
8	findFlights(string departure = "", string destination = "", string departureTime = "", string stop = "", int flightNo = -1)	Повертає список рейсів, які підходять під задані критерії
9	sortFlights(list<Flight> flightsList, int filterKey)	Сортує переданий список рейсів за вказаним ключем
10	safeInput()	Безпечно введення цілого числа із перевіркою
11	fromListToString(list<string> stops)	Перетворює список зупинок у один рядок для запису в “flights.csv”
12	fromStringToList(string stopsRaw)	Розбиває рядок зупинок на список окремих значень.
13	fromStringToPair(string line)	Перетворює рядок маршруту на пару “місце відправлення – місце прибуття”
14	add() override	Додає рейс до списку
15	remove() override	Видаляє рейс із БД
16	edit() override	Редагує рейс
17	search() override	Шукає рейси за заданим критерієм
18	searchMultiple()	Шукає рейси за кількома критеріями
19	sortData() override	Сортує рейси за заданими критеріями
20	filter() override	Фільтрує рейси за заданою умовою
21	compareByNo(const Flight& a, const Flight& b)	Компаратор для сортування за номером рейсу
22	compareByDeparture(const Flight& a, const Flight& b)	Компаратор для сортування за місцем відправки

23	compareByDestination(const Flight& a, const Flight& b)	Компаратор для сортування за місцем прибуття
24	compareByDeaprtureTime(const Flight& a, const Flight& b)	Компаратор для сортування за часом відправки
25	compareByDuration(const Flight& a, const Flight& b)	Компаратор для сортування за тривалістю польоту
26	compareBySeats(const Flight& a, const Flight& b)	Компаратор для сортування за кількістю вільних місць
27	isValidTimeFormat(const string& time)	Перевіряє чи коректно заданий часовий формат
28	isValidRoute(const string& newRoute)	Перевіряє чи коректно заданий маршрут
29	isValidNo(const int& newNo)	Перевіряє чи коректно заданий номер рейсу
30	isNumeric(const string& s)	Перевіряє чи є рядок числом

Таблиця 3 – Документування класу *UserManager*

Характеристика		Опис
Назва класу		UserManager
Опис класу		Клас нащадок, клас менеджер для керування колекцією об'єктів User. Спрямований на реєстрування, редагування, видалення, пошук та виконання авторизації користувачів.
Залежності від інших класів		IManagable – клас інтерфейс – та FileHandler – абстрактний клас
Опис атрибутів (полів)		userList(список усіх користувачів завантажених із файлу “users.txt”)
Опис методів		
1	loadFromFile()	Зчитує всі рейси із TXT файлу “users.txt” та зберігає їх у список userList
2	saveToFile()	Записує актуальний список користувачів userList у TXT файл “users.txt”
3	showUserList()	Виводить список усіх зареєстрованих користувачів
4	showAdminHelp()	Виводить інструкцію функціональних можливостей для адміна
5	showUserHelp()	Виводить інструкцію функціональних можливостей для користувача
6	find(const string& username)	Пошук користувача за логіном у списку userList. Повертає вказівник на знайденого користувача або nullptr, якщо такого логіна немає
7	add() override	Додає нового користувача
8	edit() override	Редагує дані користувача
9	remove() override	Видаляє користувача із списку
10	search() override	Шукає користувача за наймом
11	sortData() override	Сортує список користувачів за певним критерієм
12	filter() override	Фільтрує користувачів
13	login(const string& username, const string& password, const string& roleOut)	Перевіряє логін і пароль, у разі успіху повертає роль користувача.

Таблиця 4 – Документування класу *Flight*

Характеристика		Опис
Назва класу		Flight
Опис класу		Клас що представляє собою авіарейс
Залежності від інших класів		—
Опис атрибутів (полів)		raceNo(номер рейсу), route(маршрут), stops(зупинки), departureTime(час відправки), flightTime(тривалість польоту), availableSeats(кількість доступних місць)
Опис методів		
1	hasStop(const string& stopName)	Перевіряє чи міститься назва зупинки у stops
2	getRouteString() const	Повертає маршрут у вигляді одного рядка
3	getTotalStops() const	Повертає загальну кількість проміжних зупинок
4	isFull() const	Повертає true, якщо кількість вільних місць – 0
5	toCSV() const	Формує рядок у форматі CSV для збереження рейсу у файл

Таблиця 5 – Документування класу *User*

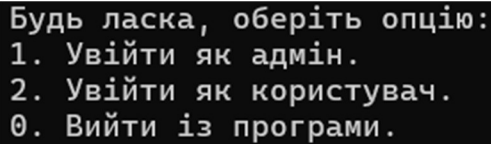
Характеристика		Опис
Назва класу		User
Опис класу		Клас що представляє собою користувача
Залежності від інших класів		Оголошує в собі class FlighManager & class UserManager для використання їх у власних методах.
Опис атрибутів (полів)		username(ім'я користувача),password(пароль користувача), role(роль користувача)
Опис методів		
1	buyTicket(FlightManager& flightManager)	Ініціює процес купівлі квитка та оформлення посадкової відомості
2	changePassword()	Змінює пароль користувача
3	changeUsername()	Змінює ім'я користувача
4	showProfile()	Показує профіль користувача
5	viewBoardingPass()	Виводить історію придбаних квитків користувача

Таблиця 6 – Документування класу *FileHandler*

Характеристика		Опис
Назва класу		FileHandler
Опис класу		Абстрактний клас для роботи із файлами
Залежності від інших класів		—
Опис атрибутів (полів)		filepath(шлях до файлу)
Опис методів		
1	virtual loadFromFile()	Функція для читання даних із файлів
2	virtual saveToFile()	Функція для збереження даних у файл
3	fileExists() const	Перевіряє чи файл існує
4	countOfElemnts() const	Рахує кількість елементів у файлі
5	validateFileFormat(const string& expectedExpression) const	Перевіряє розширення файлу

2.3. Опис програмної системи

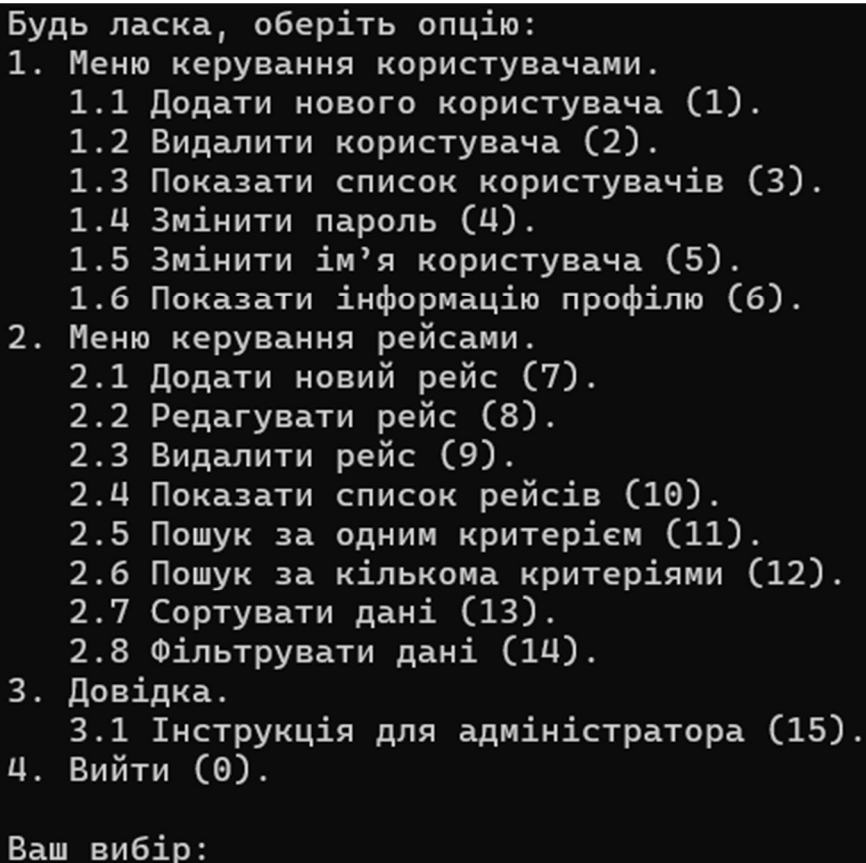
Одразу після запуску програми користувачу виводиться початкове меню авторизації (рис. 6)



```
Будь ласка, оберіть опцію:  
1. Увійти як адмін.  
2. Увійти як користувач.  
0. Вийти із програми.
```

Рисунок 6 – Меню авторизації

У разі успішної авторизації під профілем адміністратора, користувач побачить меню, яке в загальному складається із 16-ти пунктів (рис. 7). Адміністратор має змогу додавати, видаляти, змінювати інформацію про користувачів та рейси.



```
Будь ласка, оберіть опцію:  
1. Меню керування користувачами.  
  1.1 Додати нового користувача (1).  
  1.2 Видалити користувача (2).  
  1.3 Показати список користувачів (3).  
  1.4 Змінити пароль (4).  
  1.5 Змінити ім'я користувача (5).  
  1.6 Показати інформацію профілю (6).  
2. Меню керування рейсами.  
  2.1 Додати новий рейс (7).  
  2.2 Редагувати рейс (8).  
  2.3 Видалити рейс (9).  
  2.4 Показати список рейсів (10).  
  2.5 Пошук за одним критерієм (11).  
  2.6 Пошук за кількома критеріями (12).  
  2.7 Сортувати дані (13).  
  2.8 Фільтрувати дані (14).  
3. Довідка.  
  3.1 Інструкція для адміністратора (15).  
4. Вийти (0).  
  
Ваш вибір:
```

Рисунок 7 – Адміністративне меню

Обравши пункт 15 в адміністративному меню, програма виведе користувачеві інструкцію адміністративного меню, де буде надано інформацію про всі функціональні можливості адміністратора(рис. 8).

```
===== МЕНЮ ДОВІДКИ АДМІНІСТРАТОРА =====
Як адміністратор, ви маєте повний доступ до керування користувачами та рейсами.

->>>КЕРУВАННЯ КОРИСТУВАЧАМИ<<<-:
(1) Додати нового користувача – створює новий обліковий запис із логіном і паролем.
(2) Видалити користувача – видаляє існуючого користувача (крім адміністратора).
(3) Показати список користувачів – відображає всіх зареєстрованих користувачів та їх ролі.
(4) Змінити пароль – змінює ваш власний пароль адміністратора.
(5) Змінити ім'я користувача – змінює ваш логін адміністратора.
(6) Показати інформацію профілю – відображає ваш поточний профіль адміністратора.

->>>КЕРУВАННЯ РЕЙСАМИ<<<-:
(7) Додати новий рейс – створює новий рейс із маршрутом, зупинками та кількістю місць.
(8) Редагувати рейс – змінює дані рейсу (маршрут, час, місця тощо).
(9) Видалити рейс – видаляє конкретний рейс за його номером.
(10) Показати список рейсів – відображає всі наявні рейси.
(11) Пошук – знаходить рейси за одним критерієм (наприклад, пунктом призначення або номером).
(12) Багатокритеріальний пошук – поєднує кілька критеріїв для детального пошуку.
(13) Сортувати дані – сортує рейси за тривалістю, часом вильоту або кількістю місць.
(14) Фільтрувати дані – показує лише рейси, які відповідають заданим умовам.

->>>ІНШЕ<<<-:
(15) Показати довідку (це меню).
(0) Вихід до головного меню.
=====
```

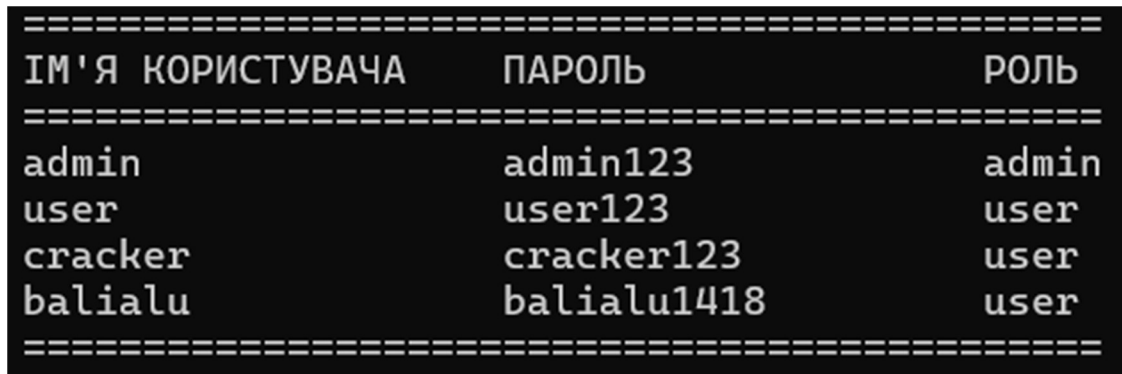
Рисунок 8 – Інструкція адміністратора

Обравши пункт 1 користувач матиме змогу додати нового користувача, ввівши його логін та пароль(рис. 9).

```
Введіть ім'я нового користувача: balialu
Введіть пароль нового користувача: balialu1418
[ІНФО] Користувача було успішно додано!.
```

Рисунок 9 – Додавання нового користувача

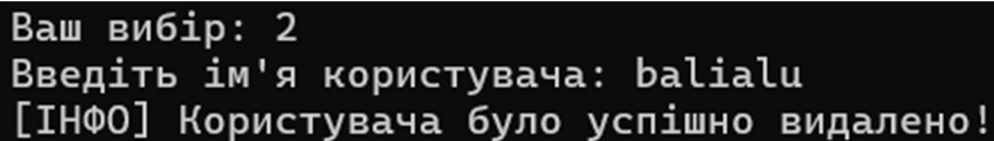
Обравши пункт 3 програма виведе в консоль список усіх зареєстрованих користувачів(рис. 10).



ІМ'Я КОРИСТУВАЧА	ПАРОЛЬ	РОЛЬ
admin	admin123	admin
user	user123	user
cracker	cracker123	user
balialu	balialu1418	user

Рисунок 10 – Вивід усіх зареєстрованих користувачів

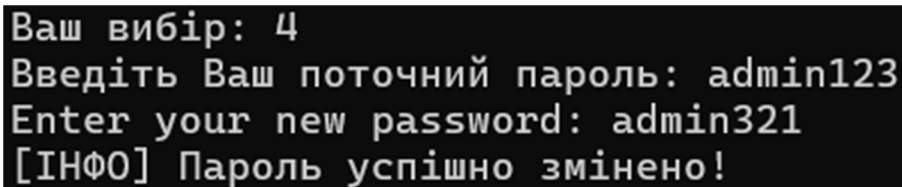
Обравши пункт 2 користувач матиме змогу видалити профіль одного із користувачів із системи, ввівши його логін(рис. 11).



```
Ваш вибір: 2
Введіть ім'я користувача: balialu
[ІНФО] Користувача було успішно видалено!
```

Рисунок 11 – Видалення профілю користувача із системи

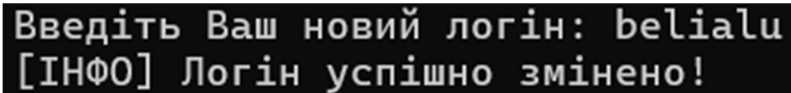
Обравши пункт 4 користувач матиме змогу змінити свій поточний пароль(рис. 12).



```
Ваш вибір: 4
Введіть Ваш поточний пароль: admin123
Enter your new password: admin321
[ІНФО] Пароль успішно змінено!
```

Рисунок 12 – Зміна паролю користувача

Обравши пункт 5 користувач зможе змінити власне ім'я(рис. 13).



```
Введіть Ваш новий логін: belialu  
[ІНФО] Логін успішно змінено!
```

Рисунок 13 – Зміна імені користувача

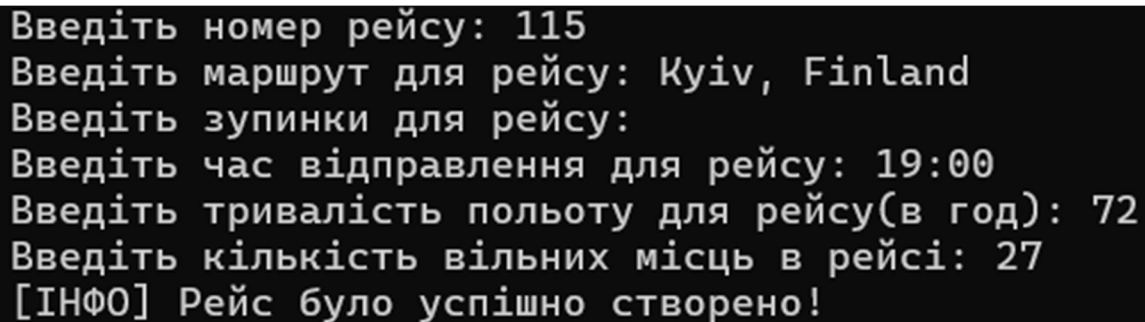
Обравши пункт 6 користувачу виведеться вся інформація про поточний профіль(рис. 14)



```
Логін:          belialu  
Роль:           admin
```

Рисунок 14 – Вивід профільної інформації

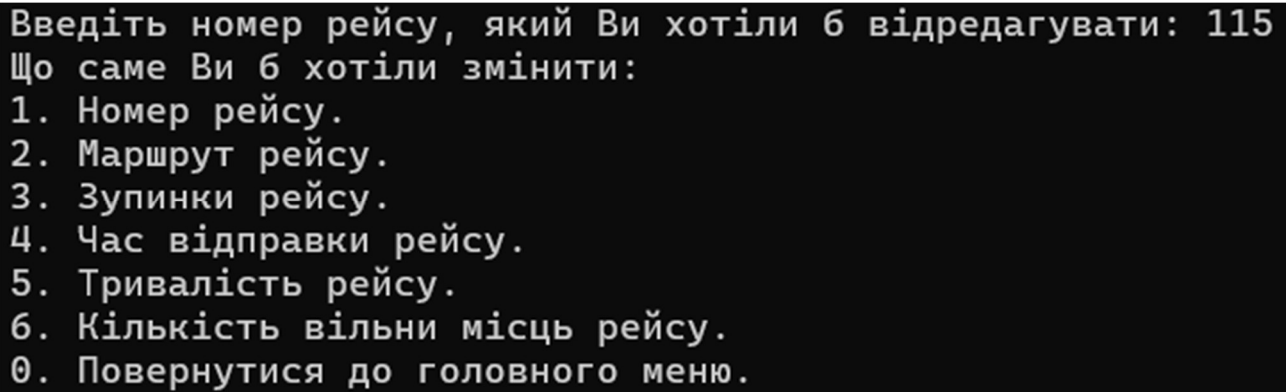
Обравши пункт 7 користувач матиме змогу додати новий рейс. Після заповнення усіх полів новостворений об'єкт буде додано до спільної бази даних(рис. 15).



```
Введіть номер рейсу: 115  
Введіть маршрут для рейсу: Kyiv, Finland  
Введіть зупинки для рейсу:  
Введіть час відправлення для рейсу: 19:00  
Введіть тривалість польоту для рейсу(в год): 72  
Введіть кількість вільних місць в рейсі: 27  
[ІНФО] Рейс було успішно створено!
```

Рисунок 15 – Створення нового рейсу

Обравши пункт 8 та ввівши номер рейсу(рис.16), користувач зможе змінювати будь-яке поле даних, що містить в собі рейс простим введенням бажаного для зміни параметру(рис. 17).



```
Введіть номер рейсу, який Ви хотіли б відредагувати: 115
Що саме Ви б хотіли змінити:
1. Номер рейсу.
2. Маршрут рейсу.
3. Зупинки рейсу.
4. Час відправки рейсу.
5. Тривалість рейсу.
6. Кількість вільних місць рейсу.
0. Повернутися до головного меню.
```

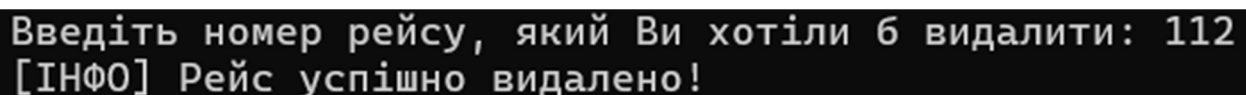
Рисунок 16 – Вибір рейсу для зміни параметрів



```
Введіть новий номер рейсу: 112
[ІНФО] Успішно змінено!
```

Рисунок 17 – Зміна номеру рейсу

Обравши пункт 9 користувач матиме змогу видалити обраний рейс, ввівши його номер(рис. 18).



```
Введіть номер рейсу, який Ви хотіли б видалити: 112
[ІНФО] Рейс успішно видалено!
```

Рисунок 18 – Видалення рейсу за його номером

Обравши пункт 10 користувачу на консоль виведе повний список усіх рейсів зареєстрованих у системі(рис. 19).

%РЕЙСУ.	МАРШРУТ	ЗУПИНКИ	ЧАС ВІДПРАВКИ	ЧАС ПОЛЬОТУ(ГОД)	ДОСТУПНІ МІСЦЯ
101	Kyiv - London	Warsaw-Berlin	08:30	180	42
102	Kyiv - Paris	Prague	09:15	200	15
103	Kyiv - New York	Warsaw-Reykjavik	10:00	540	60
104	Kyiv - Rome	Budapest	20:30	160	20
105	Kyiv - Istanbul	none	12:10	120	5
106	Kyiv - Warsaw	none	13:00	90	33
107	Kyiv - Vienna	Budapest-Bratislava	14:40	150	27
108	Kyiv - Barcelona	Milan-Nice	15:15	210	18
109	Kyiv - Oslo	Stockholm	16:00	240	22
110	Kyiv - Berlin	none	17:00	130	44
111	Kyiv - Warsaw	Lviv	18:00	10	87

В загальному 11 рейсів знайдено.

Рисунок 19 – Вивід усіх рейсів в системі

Обравши пункт 11 користувач матиме змогу вивести список усіх елементів списку із рейсами, які підпадають під вказаний параметр(рис. 20).

ПОШУК за:

1. Точкою відправки.
2. Точкою прибуття.
3. Часом відправки.
4. Номером рейсу.
5. Зупинками.
0. Повернутись до головного меню.

Ваш вибір: 2

Введіть точку прибуття: Warsaw

%РЕЙСУ.	МАРШРУТ	ЗУПИНКИ	ЧАС ВІДПРАВКИ	ЧАС ПОЛЬОТУ(ГОД)	ДОСТУПНІ МІСЦЯ
106	Kyiv - Warsaw	none	13:00	90	33
111	Kyiv - Warsaw	Lviv	18:00	10	87

Рисунок 20 – Пошук елемента за вказаним параметром

Обравши пункт 12 користувач зможе шукати потрібний рейс за кількома параметрами(рис. 21). Якщо для пошуку користувачу необхідно заповнити, до прикладу, лише два параметри, то він може заповнити лише необхідні йому поля, при цьому залишивши непотрібні поля пустими(рис. 22).

```

Введіть точку відправки(або залиште поле пустим): Kyiv
Введіть точку прибуття(або залишт поле пустим): Warsaw
Введіть час відправки(або залиште поле пусим): 18:00
Введіть пункти проміжної зупинки(або залиште поле пусим): Lviv
Введіть номер рейсу(або залиште поле пустим): 111

```

%РЕЙСУ.	МАРШРУТ	ЗУПИНКИ	ЧАС ВІДПРАВКИ	ЧАС ПОЛЬОТУ(ГОД)	ДОСТУПНІ МІСЦЯ
111	Kyiv - Warsaw	Lviv	18:00	10	87

Рисунок 21 – Пошук за всіма критеріями

```

Введіть точку відправки(або залиште поле пустим):
Введіть точку прибуття(або залишт поле пустим): Warsaw
Введіть час відправки(або залиште поле пусим): 18:00
Введіть пункти проміжної зупинки(або залиште поле пусим):
Введіть номер рейсу(або залиште поле пустим):

```

%РЕЙСУ.	МАРШРУТ	ЗУПИНКИ	ЧАС ВІДПРАВКИ	ЧАС ПОЛЬОТУ(ГОД)	ДОСТУПНІ МІСЦЯ
111	Kyiv - Warsaw	Lviv	18:00	10	87

Рисунок 22 – Пошук за двома із п'яти критерій

Обравши 13 пункт користувач зможе вивести в консоль відсортовані дані, в залежності від обраного ключа(рис. 23).

```

Сортувати за:
1. Номером рейсу.
2. Точкою відправки.
3. Точкою прибуття.
4. Часом відправки.
5. Тривалістю польоту.
6. Кількістю вільних місць.
5

```

%РЕЙСУ.	МАРШРУТ	ЗУПИНКИ	ЧАС ВІДПРАВКИ	ЧАС ПОЛЬОТУ(ГОД)	ДОСТУПНІ МІСЦЯ
111	Kyiv - Warsaw	Lviv	18:00	10	87
106	Kyiv - Warsaw	none	13:00	90	33
105	Kyiv - Istanbul	none	12:10	120	5
110	Kyiv - Berlin	none	17:00	130	44
107	Kyiv - Vienna	Budapest-Bratislava	14:40	150	27
104	Kyiv - Rome	Budapest	20:30	160	20
101	Kyiv - London	Warsaw-Berlin	08:30	180	42
102	Kyiv - Paris	Prague	09:15	200	15
108	Kyiv - Barcelona	Milan-Nice	15:15	210	18
109	Kyiv - Oslo	Stockholm	16:00	240	22
103	Kyiv - New York	Warsaw-Reykjavik	10:00	540	60

Рисунок 23 – Сортування за тривалістю польоту

Обравши пункт 14 користувач зможе відфільтрувати рейси в залежності від ключа фільтрування(рис. 24).

Фільтрувати рейси за (залиште поля пустими, щоб пропустити):
Точкою відправки:
Точкою прибуття: London
Часом відправки:
Пунктом проміжної зупинки:
Номером рейсу:

%РЕЙСУ.	МАРШРУТ	ЗУПИНКИ	ЧАС ВІДПРАВКИ	ЧАС ПОЛЬОТУ(ГОД)	ДОСТУПНІ МІСЦЯ
101	Kyiv - London	Warsaw-Berlin	08:30	180	42

Рисунок 24 – Фільтрування за місцем прибуття

Після успішної авторизації під обліковим записом звичайного користувача на екрані з'являється меню, що загалом містить 12 пунктів (рис. 25). У цьому меню користувач може переглядати список рейсів, виконувати пошук, сортування та фільтрацію, а також оформлювати купівлю квитка. Окрім роботи з рейсами, передбачено й керування власним профілем: можна змінити логін і пароль, переглянути основну інформацію про себе та ознайомитися з історією власних посадкових талонів.

Будь ласка, оберіть опцію:

1. Меню рейсів.
 - 1.1 Показати всі рейси (1).
 - 1.2 Пошук за одним критерієм (2).
 - 1.3 Пошук за кількома критеріями (3).
 - 1.4 Сортувати рейси (4).
 - 1.5 Фільтрувати рейси (5).
 - 1.6 Придбати квиток (6).
2. Меню профілю.
 - 2.1 Змінити ім'я користувача (7).
 - 2.2 Змінити пароль (8).
 - 2.3 Показати інформацію профілю (9).
 - 2.4 Переглянути історію посадкових талонів (10).
3. Довідка.
 - 3.1 Інструкція користувача (11).
4. Вийти (0).

Ваш вибір:

Рисунок 25 – меню звичайного користувача

Обравши пункт 11 користувач зможе відкрити інструкцію користувачького меню, де буде надано інформацію про всі функціональні можливості користувача(рис. 26).

```
===== МЕНЮ ДОВІДКИ КОРИСТУВАЧА =====
Як користувач, ви можете переглядати рейси, купувати квитки та керувати своїм профілем.

->>>ОПЕРАЦІЇ З РЕЙСАМИ<<<-:
(1) Показати рейси - відображає всі доступні рейси.
(2) Пошук - знаходить рейси за одним критерієм (наприклад, пунктом призначення чи часом).
(3) Багатокритеріальний пошук - дозволяє здійснити пошук за кількома параметрами (маршрут, зупинка, час тощо).
(4) Сортувати - впорядковує рейси за часом, тривалістю або кількістю місць.
(5) Фільтрувати - показує лише ті рейси, які відповідають заданим параметрам.
(6) Придбати квиток - дає змогу купити квиток на обраний рейс.

->>>ОПЕРАЦІЇ З ПРОФІЛЕМ<<<-:
(7) Змінити ім'я користувача - оновлює ваш логін облікового запису.
(8) Змінити пароль - оновлює пароль вашого облікового запису.
(9) Показати інформацію профілю - відображає поточні дані вашого профілю.
(10) Переглянути історію посадкових талонів - показує список усіх ваших придбаних квитків.

->>>ІНШЕ<<<-:
(11) Показати довідку (це меню).
(0) Вихід із облікового запису.
=====
```

Рисунок 26 – Інструкція користувача

Обравши пункт 6 користувач може придбати квиток ввівши потрібний йому пункт призначення(рис. 27). Після цього користувачу доведеться ввести кількість квитків, які той хоче придбати(рис. 28).

```
Введіть пункт призначення: Warsaw
Найближчий рейс до вказаного місця знайдено:
=====
%РЕЙСУ.  МАРШРУТ                ЗУПИНКИ                ЧАС ВІДПРАВКИ        ЧАС ПОЛЬОТУ(ГОД)    ДОСТУПНІ МІСЦЯ
=====
106      Kyiv - Warsaw                none                    13:00                  90                   33
=====
```

Рисунок 27 – Пошук найближчого рейсу до вказаного місця

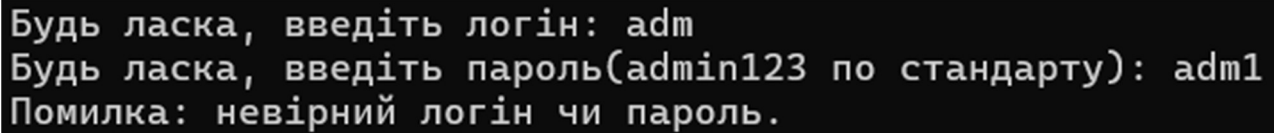
```
Будь ласка, введіть кількість квитків, яку Ви хотіли б придбати: 1
Будь ласка, підтвердьте Вашу покупку (y/n): y
[ІНФО] Дякуємо за Вашу покупку!
```

Рисунок 28 – Процес купівлі квитка

2.4. Тестування програмної системи

Тест авторизації користувача

При введенні некоректних даних чи даних про неіснуючого користувача, програма видаватиме помилку(рис. 29).

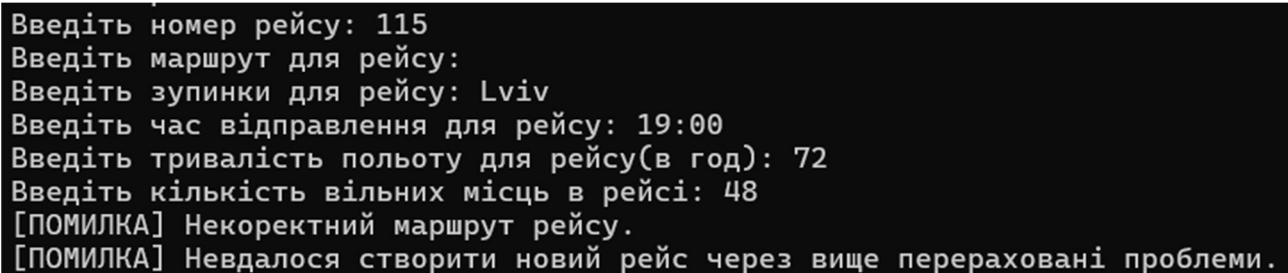


```
Будь ласка, введіть логін: adm
Будь ласка, введіть пароль(admin123 по стандарту): adm1
Помилка: невірний логін чи пароль.
```

Рисунок 29 – Некоректно введена інформація при авторизації

Тест на додавання нового об'єкта

При введенні некоректних значень в параметрах створюваного об'єкту, програма видаватиме помилку(рис. 30).

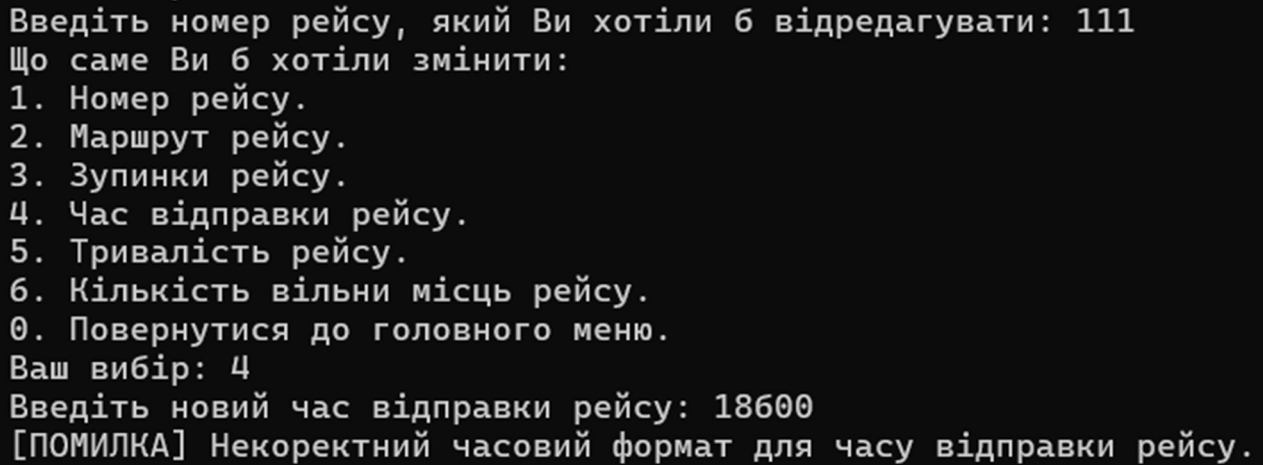


```
Введіть номер рейсу: 115
Введіть маршрут для рейсу:
Введіть зупинки для рейсу: Lviv
Введіть час відправлення для рейсу: 19:00
Введіть тривалість польоту для рейсу(в год): 72
Введіть кількість вільних місць в рейсі: 48
[ПОМИЛКА] Некоректний маршрут рейсу.
[ПОМИЛКА] Невдалося створити новий рейс через вище перераховані проблеми.
```

Рисунок 30 – Некоректно введений параметр при
Додаванні об'єкту

Тест на редагування даних об'єкта

При введенні некоректних даних для зміни деякого параметру рейсу, програма видаватиме помилку(рис. 31).

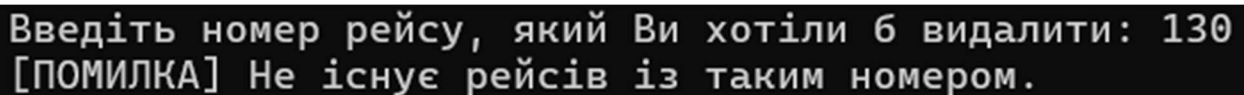


```
Введіть номер рейсу, який Ви хотіли б відредагувати: 111
Що саме Ви б хотіли змінити:
1. Номер рейсу.
2. Маршрут рейсу.
3. Зупинки рейсу.
4. Час відправки рейсу.
5. Тривалість рейсу.
6. Кількість вільних місць рейсу.
0. Повернутися до головного меню.
Ваш вибір: 4
Введіть новий час відправки рейсу: 18600
[ПОМИЛКА] Некоректний часовий формат для часу відправки рейсу.
```

Рисунок 31 – Некоректно введений параметр

Тест на видалення об'єкта

При спробі видалити рейс із неіснуючим номером, програма видає помилку(рис. 32).



```
Введіть номер рейсу, який Ви хотіли б видалити: 130
[ПОМИЛКА] Не існує рейсів із таким номером.
```

Рисунок 32 – Помилка при видаленні рейсу
із неіснуючим номером

2.5. Програмні засоби розробки

Розробка реалізована засобами середовища Visual Studio мовою C++. Дане середовище є зручним у використанні для швидкого та якісного створення консольних і віконних додатків під ОС Windows.

Microsoft – це компанія, яка розробляє високопродуктивні інтегровані середовища розробки (IDE) для різних мов програмування, зокрема Visual Studio для C++. Це програмне забезпечення надає широкий набір інструментів, що суттєво спрощують процес написання, налагодження та тестування програм: підсвічування синтаксису, автодоповнення коду, вбудований відлагоджувач, зручне керування проєктами. Для автоматизації збирання застосунку використовуються вбудовані засоби керування проєктами Visual Studio, які дозволяють легко конфігурувати параметри компіляції та адаптувати проєкт до різних конфігурацій середовища.

ВИСНОВКИ

У ході виконання курсового проєкту було розроблено консольну інформаційну систему для роботи з базою даних авіарейсів.

У програмі реалізовано:

- авторизацію користувачів із розмежуванням прав доступу;
- окремі класи для подання рейсів і користувачів;
- класи-менеджери які відповідають за виконання базових операцій із об'єктами;
- додатковий абстрактний клас який спрощує завантаження й збереження інформації у файли форматів .csv та .txt;

У процесі розробки було закріплено навички:

- проєктування структури програмного забезпечення;
- роботи із файлами;
- обробки помилкового вводу;
- організації діалогу з користувачем через текстове меню;

В майбутньому розроблена система може бути розширена. Перспекативними напрямками для розвитку може виступати розробка графічного інтерфейсу користувача(GUI), підтримка додаткових форматів збереження даних і інтеграція з мережею для оновлення інформації про рейси в режимі реального часу.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) Cplusplus.com - <https://cplusplus.com/reference/stl>
- 2) Acode.com.ua - <https://acode.com.ua/urok-220-bazovyj-fajlovyj-vvid-i-vyvid>
- 3) Doxygen User Manual - https://www.doxygen.nl/manual/doxygen_usage.html
- 4) Geeksforgeeks.org - <https://www.geeksforgeeks.org/cpp/comparator-in-cpp>
- 5) Bjarne Stroustrup, "The C++ Programming Language," 4th Edition, Addison-Wesley, 2013.
- 6) Bjarne Stroustrup, "Programming: Principles and Practise Using C++," 3rd Edition, 2024.
- 7) Herb Sutter, "Exceptional C++: 47 Engineering Puzzles, Programming Problems, and Solutions," Addison-Wesley, 2000.c

ДОДАТКИ

ДОДАТОК А. Скролінг (текст) програми

Програмний модуль «course-project»

– Вміст course-project.cpp

```
// COURSE_PROJECT.cpp : This file contains the 'main' function. Program execution begins and ends there.
//
```

```
#include "UserManager.h"
#include "FlightManager.h"
#include "Flight.h"
#include "User.h"
#include <iostream>
#include <string>
#include <locale>
#include <windows.h>

using namespace std;

/// SAFE INPUT ///
int safeInput() {
    int value;

    try {
        cin >> value;

        if (cin.fail()) {
            cin.clear();
            cin.ignore((std::numeric_limits<std::streamsize>::max)(), '\n');
            throw runtime_error("[ПОМИЛКА] Некоректно введено значення\
                               (значення має бути числовим значенням int).\n");
        }

        cin.ignore((std::numeric_limits<std::streamsize>::max)(), '\n');
        return value;
    }
    catch (const runtime_error& e) {
        cout << e.what() << endl;
        return -1;
    }
}

bool checkValue(const string& input) {
    if (input.empty() && input.length() > 1) return false;

    try {
        int value = stoi(input);
        return (value >= 0 && value <= 2);
    }
    catch (const invalid_argument&) {
        return false;
    }
    catch (const out_of_range&) {
        return false;
    }
}

/// PROGRAM'S MAIN MENU ///
void mainInterface() {
    UserManager userManager("users.txt");
    FlightManager flightManager("flights.csv");
    userManager.loadFromFile();
    flightManager.loadFromFile();

    cout << "Вітаємо, користувачу!" << endl;
    while (true) {
        cout << "Будь ласка, оберіть опцію:" << endl;
        cout << "1. Увійти як адмін." << endl;
        cout << "2. Увійти як користувач." << endl;
        cout << "0. Вийти із програми." << endl;
        int input = safeInput();
```

```

if (input == -1) {
    continue;
}

if (input > 2 || input < 0) {
    cout << "Помилка: Невірна опція!" << endl;
    continue;
}

if (input == 0) {
    cout << "До зустрічі!" << endl;
    break;
}

string username, password, role;
cout << "Будь ласка, введіть логін: ";
getline(cin, username);
if (input == 1) cout << "Будь ласка, введіть пароль(admin123 по стандарту): ";
else if (input == 2) cout << "Будь ласка, введіть пароль(user123 по стандарту): ";
getline(cin, password);

if (userManager.login(username, password, role)) {
    User user(username, password, role);
    bool session = true;
    cout << "Вхід успішний!\n" << endl;
    while (session) {
        cout << "Будь ласка, оберіть опцію: " << endl;
        int choice;
        if (role == "admin") {
            cout << "1. Меню керування користувачами." << endl;
            cout << "    1.1 Додати нового користувача (1)." << endl;
            cout << "    1.2 Видалити користувача (2)." << endl;
            cout << "    1.3 Показати список користувачів (3)." << endl;
            cout << "    1.4 Змінити пароль (4)." << endl;
            cout << "    1.5 Змінити ім'я користувача (5)." << endl;
            cout << "    1.6 Показати інформацію профілю (6)." << endl;
            cout << "2. Меню керування рейсами." << endl;
            cout << "    2.1 Додати новий рейс (7)." << endl;
            cout << "    2.2 Редагувати рейс (8)." << endl;
            cout << "    2.3 Видалити рейс (9)." << endl;
            cout << "    2.4 Показати список рейсів (10)." << endl;
            cout << "    2.5 Пошук за одним критерієм (11)." << endl;
            cout << "    2.6 Пошук за кількома критеріями (12)." << endl;
            cout << "    2.7 Сортувати дані (13)." << endl;
            cout << "    2.8 Фільтрувати дані (14)." << endl;
            cout << "3. Довідка." << endl;
            cout << "    3.1 Інструкція для адміністратора (15)." << endl;
            cout << "4. Вийти (0)." << endl << endl;
            cout << "Ваш вибір: ";
            choice = safeInput();
            if (choice == -1) {
                continue;
            }
            switch (choice) {
                case 1:
                    userManager.add();
                    break;
                case 2:
                    userManager.remove();
                    break;
                case 3:
                    userManager.showUserList();
                    break;
                case 4:
                    user.changePassword();
                    break;
                case 5:
                    user.changeUsername(userManager);
                    break;
                case 6:
                    user.showProfile();
                    break;
                case 7:
                    flightManager.add();
                    break;
                case 8:
                    flightManager.edit();
                    break;
            }
        }
    }
}

```

```

case 9:
    flightManager.remove();
    break;
case 10:
    flightManager.showFlightList(flightManager.getFlights());
    break;
case 11:
    flightManager.search();
    break;
case 12:
    flightManager.searchMultiple();
    break;
case 13:
    flightManager.sortData();
    break;
case 14:
    flightManager.filter();
    break;
case 15:
    userManager.showAdminHelp();
    break;
case 0:
    session = false;
    break;
default:
    cout << "Помилка: Невірна опція!\n" << endl;
    break;
}

}
if (role == "user") {
    cout << "1. Меню рейсів." << endl;
    cout << "    1.1 Показати всі рейси (1)." << endl;
    cout << "    1.2 Пошук за одним критерієм (2)." << endl;
    cout << "    1.3 Пошук за кількома критеріями (3)." << endl;
    cout << "    1.4 Сортувати рейси (4)." << endl;
    cout << "    1.5 Фільтрувати рейси (5)." << endl;
    cout << "    1.6 Придбати квиток (6)." << endl;
    cout << "2. Меню профілю." << endl;
    cout << "    2.1 Змінити ім'я користувача (7)." << endl;
    cout << "    2.2 Змінити пароль (8)." << endl;
    cout << "    2.3 Показати інформацію профілю (9)." << endl;
    cout << "    2.4 Переглянути історію посадкових талонів (10)." << endl;
    cout << "3. Довідка." << endl;
    cout << "    3.1 Інструкція користувача (11)." << endl;
    cout << "4. Вийти (0)." << endl << endl;
    cout << "Ваш вибір: ";

    int choice = safeInput();
    switch (choice) {
    case 0:
        session = false;
        break;
    case 1:
        flightManager.showFlightList(flightManager.getFlights());
        break;
    case 2:
        flightManager.search();
        break;
    case 3:
        flightManager.searchMultiple();
        break;
    case 4:
        flightManager.sortData();
        break;
    case 5:
        flightManager.filter();
        break;
    case 6:
        user.buyTicket(flightManager);
        break;
    case 7:
        user.changeUsername(userManager);
        break;
    case 8:
        user.changePassword();
        break;
    case 9:

```

```

        user.showProfile();
        break;
    case 10:
        user.viewBoardingPass();
        break;
    case 11:
        userManager.showUserHelp();
        break;
    default:
        cout << "Помилка: Невірна опція!\n" << endl;
        break;
    }
}

}

}
else { cout << "Помилка: невірний логін чи пароль.\n" << endl; }

}
return;
}

int main()
{
    setlocale(LC_ALL, "ukr");
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);

    mainInterface();
}

```

Програмний модуль «IManagable»

— Вміст IManagable.h

```

#pragma once
#include <string>
#include <iostream>
#include <list>
#include <iomanip>
#include <sstream>
#include <fstream>
#include <utility>
#include <regex>
#include <cctype>
#include <algorithm>

/**
 * @interface IManagable
 * @brief Абстрактний інтерфейс для керування об'єктами системи.
 * @details
 * Інтерфейс `IManagable` визначає базовий набір методів,
 * які повинні бути реалізовані будь-яким класом, що підтримує
 * операції керування даними (наприклад, додавання, редагування, видалення, пошук тощо).
 * Використовується як базовий клас для:
 * - @ref FlightManager - керування рейсами;
 * - @ref UserManager - керування користувачами.
 */
class IManagable
{
public:
    /**
     * @brief Віртуальний деструктор.
     * @details Забезпечує коректне звільнення ресурсів у похідних класах.
     */
    virtual ~IManagable() = default;

    /// ===== MAIN FUNCTIONALITY ===== ///
    /**
     * @brief Додає новий об'єкт.
     * @details Реалізується у похідних класах для створення нового запису (рейсу, користувача тощо).
     */
    virtual void add() = 0;

    /**
     * @brief Редагує існуючий об'єкт.

```

```

    * @details Використовується для зміни даних наявного запису.
    */
    virtual void edit() = 0;

    /**
     * @brief Видаляє об'єкт.
     * @details Реалізується для видалення записів із колекції або файлу.
     */
    virtual void remove() = 0;

    /**
     * @brief Виконує пошук об'єктів.
     * @details Дає змогу знаходити об'єкти за одним критерієм (наприклад, номером рейсу або іменем користувача).
     */
    virtual void search() = 0;

    /**
     * @brief Сортиє об'єкти за визначеним параметром.
     * @details Використовується для впорядкування даних (наприклад, за іменем, датою чи номером).
     */
    virtual void sortData() = 0;

    /**
     * @brief Фільтрує об'єкти за кількома умовами.
     * @details Дозволяє обмежити вивід лише тими елементами, що відповідають зазначеним критеріям.
     */
    virtual void filter() = 0;
};

```

Програмний модуль «FlightManager»

– Вміст FlightManager.h

```

#pragma once
#include "IManagable.h"
#include "FileHandler.h"
#include "Flight.h"
#include "User.h"

/**
 * @class FlightManager
 * @brief Клас для керування всіма операціями з рейсами (додавання, редагування, пошук,
 * сортування, збереження у файл тощо).
 * @details
 * Клас `FlightManager` успадковує інтерфейс @ref IManagable та клас @ref FileHandler.
 * Він реалізує повний функціонал для роботи з об'єктами типу @ref Flight:
 * - Зчитування та збереження рейсів у файл.
 * - Валідація введених даних.
 * - Пошук, фільтрація, сортування рейсів.
 * - Бронювання квитків та генерація посадкових відомостей.
 * Також забезпечує користувацький інтерфейс для адміністрування рейсів.
 */
class FlightManager : public IManagable, public FileHandler
{
public:
    /// ===== CONSTRUCTORS & DESTRUCTOR ===== ///
    /**
     * @brief Конструктор за замовчуванням.
     * @details Ініціалізує об'єкт без початкових даних про рейси.
     */
    FlightManager();

    /**
     * @brief Конструктор з параметром – список рейсів.
     * @param flights Список рейсів для ініціалізації менеджера.
     */
    FlightManager(list<Flight> flights);

    /**
     * @brief Конструктор із шляхом до файлу.
     * @param filePath Шлях до файлу, з якого завантажуються дані про рейси.
     */
    FlightManager(const string& filePath);

    /**

```

```

    * @brief Копіюючий конструктор.
    * @param other Інший об'єкт FlightManager, з якого копіюються дані.
    */
    FlightManager(const FlightManager& other);

    /**
    * @brief Конструктор переміщення.
    * @param other Інший об'єкт FlightManager, з якого переміщуються дані.
    */
    FlightManager(FlightManager&& other) noexcept;

    /**
    * @brief Деструктор.
    * @details Під час знищення об'єкта автоматично зберігає усі зміни у файл.
    */
    ~FlightManager();

    /// ===== GETTERS ===== ///
    /**
    * @brief Отримує список усіх рейсів.
    * @return Список рейсів.
    */
    list<Flight> getFlights();

    /// ===== COMPARATORS ===== ///
    /**
    * @brief Порівняння рейсів за номером.
    */
    static bool compareByNo(const Flight& a, const Flight& b);

    /**
    * @brief Порівняння рейсів за пунктом відправлення.
    */
    static bool compareByDeparture(const Flight& a, const Flight& b);

    /**
    * @brief Порівняння рейсів за пунктом призначення.
    */
    static bool compareByDestination(const Flight& a, const Flight& b);

    /**
    * @brief Порівняння рейсів за часом відправлення.
    */
    static bool compareByDepartureTime(const Flight& a, const Flight& b);

    /**
    * @brief Порівняння рейсів за тривалістю польоту.
    */
    static bool compareByDuration(const Flight& a, const Flight& b);

    /**
    * @brief Порівняння рейсів за кількістю доступних місць.
    */
    static bool compareBySeats(const Flight& a, const Flight& b);

    /// ===== VALIDATION METHODS ===== ///
    /**
    * @brief Перевірка коректності формату часу (ГГ:ХХ).
    * @param time Рядок із часом.
    * @return true, якщо формат часу правильний, return false - якщо ні.
    */
    bool isValidTimeFormat(const string& time);

    /**
    * @brief Перевірка правильності формату маршруту.
    * @param newRoute Рядок із маршрутом формату "Київ-Львів".
    * @return true, якщо маршрут є коректним, return false - якщо ні.
    */
    bool isValidRoute(string newRoute);

    /**
    * @brief Перевірка унікальності номера рейсу.
    * @param newNo Номер рейсу.
    * @return true, якщо такого рейсу ще не існує.
    */
    bool isValidNo(int newNo);

```

```

/**
 * @brief Перевірка, чи відповідає рейс заданим критеріям.
 * @param flight Рейс для перевірки.
 * @param departure Місто відправлення.
 * @param destination Місто призначення.
 * @param departureTime Час відправлення.
 * @param stop Проміжна зупинка.
 * @param flightNo Номер рейсу.
 * @return true, якщо рейс відповідає усім вказаним критеріям.
 */
bool isMatchingCriteria(Flight flight, string departure = "",
    string destination = "", string departureTime = "",
    string stop = "", int flightNo = -1);

bool isNumeric(const string& s);

/// ===== DATA FILTER & SORT ===== ///
/**
 * @brief Знаходить усі рейси, що відповідають заданим критеріям.
 * @param departure Точка відправлення.
 * @param destination Точка прибуття.
 * @param departureTime Час відправлення.
 * @param stop Проміжна зупинка.
 * @param flightNo Номер рейсу.
 * @return Список знайдених рейсів.
 */
list<Flight> findFlights(string departure = "", string destination = "",
    string departureTime = "", string stop = "", int flightNo = -1);

/**
 * @brief Сортує список рейсів за заданим критерієм.
 * @param flightsList Список рейсів.
 * @param filterKey Ключ сортування (1 – номер, 2 – відправлення, тощо).
 * @return Відсортований список рейсів.
 */
list<Flight> sortFlights(list<Flight> flightsList, int filterKey, bool ascending);

/// ===== INPUT & TRANSFORM ===== ///
/**
 * @brief Безпечне введення цілого числа з перевіркою.
 * @return Коректно введене число типу int.
 */
int safeInput();

/**
 * @brief Перетворює список зупинок у рядок.
 * @param stops Список зупинок.
 * @return Рядок, який представляє список зупинок.
 */
string fromListToString(list<string> stops);

/**
 * @brief Перетворює рядок у список зупинок.
 * @param stopsRaw Рядок із зупинками.
 * @return Список зупинок.
 */
list<string> fromStringToList(string stopsRaw);

/**
 * @brief Перетворює рядок у пару міст (відправлення, прибуття).
 * @param line Рядок маршруту.
 * @return Пара міст.
 */
pair<string, string> fromStringToPair(string line);

/**
 * @brief Перетворює рядок у нижній регістр (для нечутливого до регістру пошуку).
 * @param s Рядок точки відправки/прибуття/зупинок.
 * @return Рядок точки відправки/прибуття/зупинок.
 */
static string toLower(const string& s);

/// ===== MAIN FUNCTIONALITY ===== ///
/**
 * @brief Відображає список рейсів у табличному форматі.
 * @param flightList Список рейсів для відображення.
 */
void showFlightList(list<Flight> flightList);

```



```

/**
 * @brief Знаходить найближчий рейс до заданого пункту призначення.
 * @param destination Місто призначення.
 * @return Вказівник на знайдений рейс або nullptr.
 */
Flight* findNearsetFlight(string destination);

/**
 * @brief Процес купівлі квитків користувачем.
 * @param user Об'єкт користувача, який здійснює покупку.
 */
void bookTickets(User& user);

/**
 * @brief Генерує посадковий талон (відомість) після покупки квитка.
 * @param user Користувач.
 * @param flight Обраний рейс.
 * @param ticketCount Кількість куплених квитків.
 */
void generateBoardingPass(User& user, Flight& flight, int ticketCount);

/**
 * @brief Додає новий рейс.
 */
void add() override;

/**
 * @brief Видаляє існуючий рейс.
 */
void remove() override;

/**
 * @brief Редагує дані існуючого рейсу.
 */
void edit() override;

/**
 * @brief Пошук рейсу за одним критерієм.
 */
void search() override;

/**
 * @brief Фільтрація рейсів за кількома параметрами.
 */
void filter() override;

/**
 * @brief Сорткування рейсів за вибраним параметром.
 */
void sortData() override;

/**
 * @brief Розширений пошук за кількома критеріями одночасно.
 */
void searchMultiple();

/// ===== FILE OPERATIONS ===== ///
/**
 * @brief Завантажує дані про рейси з файлу.
 * @details
 * Метод відкриває файл, шлях до якого вказано у властивості 'filepath'.
 * Якщо файл не існує – створює новий порожній файл.
 * Якщо формат файлу некоректний (не '.csv') – виводить повідомлення про помилку.
 *
 * Під час зчитування кожен рядок інтерпретується як запис рейсу та перетворюється у об'єкт
 класу @ref Flight,
 * який додається до списку рейсів 'flights'.
 *
 * @note Після успішного завантаження на екран виводиться кількість імпортованих рейсів.
 */
void loadFromFile();

/**
 * @brief Зберігає всі рейси до файлу.
 * @details
 * Метод відкриває файл, заданий у властивості 'filepath', і перезаписує його вміст.
 * Кожен рейс із списку 'flights' записується у вигляді форматowanego рядка (наприклад:

```

```

        * `Номер;Маршрут;Зупинки;Час вильоту;Тривалість;Кількість місць`).
        *
        * Якщо файл неможливо відкрити або формат неправильний – виводиться повідомлення про помилку.
        *
        * @note Викликається автоматично у деструкторі, щоб зберегти зміни перед завершенням роботи
        програми.
        */
        void saveToFile();

private:
    list<Flight> flights;
};

```

– Вміст FlightManager.cpp

```

#include "FlightManager.h"
#include <iostream>
#include <list>
#include <string>

using namespace std;

/// CONSTRUCTORS && DESTRUCTOR ///
FlightManager::FlightManager() {}
FlightManager::FlightManager(list<Flight> flights) : flights(flights) {}
FlightManager::FlightManager(const string& filePath) : FileHandler(filePath) {}
FlightManager::FlightManager(const FlightManager& other) : flights(other.flights) {}
FlightManager::FlightManager(FlightManager&& other) noexcept
    : flights(move(other.flights)) {}
FlightManager::~FlightManager() {
    saveToFile();
    cout << "[ІНФО] Дані рейсів успішно збережені до flight.csv!" << endl;
}

/// COMPARATORS ///
bool FlightManager::compareByNo(const Flight& a, const Flight& b) {
    return a.getRaceNo() < b.getRaceNo();
}

bool FlightManager::compareByDeparture(const Flight& a, const Flight& b) {
    return a.getRoute().first < b.getRoute().first;
}

bool FlightManager::compareByDestination(const Flight& a, const Flight& b) {
    return a.getRoute().second < b.getRoute().second;
}

bool FlightManager::compareByDepartureTime(const Flight& a, const Flight& b) {
    return a.getDepartureTime() < b.getDepartureTime();
}

bool FlightManager::compareByDuration(const Flight& a, const Flight& b) {
    return a.getFlightTime() < b.getFlightTime();
}

bool FlightManager::compareBySeats(const Flight& a, const Flight& b) {
    return a.getAvailableSeats() < b.getAvailableSeats();
}

/// INPUT DATA VALIDATION ///
bool FlightManager::isValidTimeFormat(const string& time) {
    regex pattern(R"(^([01][0-9]|2[0-3]):[0-5][0-9]$)");
    if (time != "" && regex_match(time, pattern)) return true;
    else {
        cout << "[ПОМИЛКА] Некоректний часовий формат для часу відправки рейсу." << endl;
        return false;
    }
}

bool FlightManager::isValidRoute(string newRoute) {
    pair<string, string> testRoute = fromStringToPair(newRoute);
    if (testRoute.first.empty() || testRoute.second.empty())

```

```

        || testRoute.first == testRoute.second) {
            cout << "[ПОМИЛКА] Некоректний маршрут рейсу." << endl;
            return false;
        }

        else return true;
    }

    bool FlightManager::isValidNo(int newNo) {
        for (auto current_flight : flights) {
            if (current_flight.getRaceNo() == newNo) {
                cout << "[ПОМИЛКА] Рейс із таким номером уже існує." << endl;
                return false;
            }
        }
        return true;
    }

    bool FlightManager::isMatchingCriteria(Flight flight, string departure,
        string destination, string departureTime, string stop, int flightNo) {

        if (!departure.empty() && toLower(flight.getRoute().first)
            != toLower(departure)) return false;
        if (!destination.empty() && toLower(flight.getRoute().second)
            != toLower(destination)) return false;
        if (!departureTime.empty() && flight.getDepartureTime()
            != departureTime) return false;
        if (flightNo != -1 && flight.getRaceNo()
            != flightNo) return false;
        if (!stop.empty()) {
            bool found = false;
            for (auto& current_stop : flight.getStops()) {
                if (toLower(current_stop) == toLower(stop)) { found = true; break; }
            }
            if (!found) return false;
        }
        return true;
    }

    bool FlightManager::isNumeric(const string& s) {
        if (s.empty()) {
            return false;
        }

        size_t start = 0;
        if (s[0] == '-' || s[0] == '+') {
            start = 1;
        }

        for (size_t i = start; i < s.length(); ++i) {
            if (isdigit(s[i])) {
                cout << "[ПОМИЛКА] Значення відправки/прибуття не може бути пустим/числом.\n" <<
endl;
                return false;
            }
        }
        return true;
    }

    /// DATA FILTER ///
    list<Flight> FlightManager::findFlights(string departure, string destination,
        string departureTime, string stop, int flightNo) {
        list<Flight> result;

        for (const auto& current_flight : flights) {
            if (isMatchingCriteria(current_flight, departure, destination,
                departureTime, stop, flightNo)) {
                result.push_back(current_flight);
            }
        }

        return result;
    }

    list<Flight> FlightManager::sortFlights(list<Flight> flightsList,
        int filterKey, bool ascending = true) {
        if (filterKey == 1) flightsList.sort(compareByNo);
        else if (filterKey == 2) flightsList.sort(compareByDeparture);
    }

```

```

        else if (filterKey == 3) flightsList.sort(compareByDestination);
        else if (filterKey == 4) flightsList.sort(compareByDepartureTime);
        else if (filterKey == 5) flightsList.sort(compareByDuration);
        else if (filterKey == 6) flightsList.sort(compareBySeats);

        if (!ascending) {
            flightsList.reverse();
        }

        return flightsList;
    }

int FlightManager::safeInput() {
    int value;
    while (true) {
        cin >> value;

        if (cin.fail()) {
            cin.clear();
            cin.ignore((std::numeric_limits<std::streamsize>::max)(), '\n');
            cout << "[ПОМИЛКА] Некоректно введено значення(число має бути числовим значенням
int):";
        }
        else {
            cin.ignore((std::numeric_limits<std::streamsize>::max)(), '\n');
            return value;
        }
    }
}

/// TRANSFORM FUNCTIONS ///
string FlightManager::fromListToString(list<string> stops) {
    string stop;
    for (auto& current_stop : stops) {
        stop += current_stop;
        if (current_stop != stops.back()) stop += "-";
    }
    return stop;
}

list<string> FlightManager::fromStringToList(string stopsRaw) {
    list<string> stops;
    const string delimiters = ";,- ";

    size_t start = 0;
    size_t end = stopsRaw.find_first_of(delimiters);

    while (end != string::npos) {
        string token = stopsRaw.substr(start, end - start);
        if (!token.empty()) stops.push_back(token);
        start = end + 1;
        end = stopsRaw.find_first_of(delimiters, start);
    }

    string lastToken = stopsRaw.substr(start);
    if (!lastToken.empty()) stops.push_back(lastToken);

    for (auto& s : stops) {
        while (!s.empty() && isspace((unsigned char)s.front())) s.erase(0, 1);
        while (!s.empty() && isspace((unsigned char)s.back())) s.pop_back();
    }

    return stops;
}

pair<string, string> FlightManager::fromStringToPair(string line) {
    pair<string, string> route;
    size_t pos = line.find_first_of("-; , ");

    if (pos != string::npos) {
        route.first = line.substr(0, pos);

        size_t startSecond = pos + 1;
        while (startSecond < line.size() && (line[startSecond] == '-'
            || line[startSecond] == ';' || line[startSecond] == ' '))
            startSecond++;
    }
}

```

```

        route.second = line.substr(startSecond);
    }
    else {
        route.first = line;
        route.second = "";
    }

    return route;
}

string FlightManager::toLower(const string& s) {
    string result = s;
    transform(result.begin(), result.end(), result.begin(),
        [](unsigned char c) {
            return static_cast<char>(tolower(c));
        });
    return result;
}

/// MAIN METHODS ///
void FlightManager::showFlightList(list<Flight> flightList) {
    if (flightList.empty()) {
        cout << "[ПОМИЛКА] Не знайдено жодних рейсів.\n" << endl;
        return;
    }
    cout << left;
    cout <<
    "=====
=====\\n";
    cout << setw(10) << "%РЕЙСУ."
        << setw(25) << "МАРШРУТ"
        << setw(25) << "ЗУПИНКИ"
        << setw(20) << "ЧАС ВІДПРАВКИ"
        << setw(20) << "ЧАС ПОЛЬОТУ(ГОД)"
        << setw(20) << "ДОСТУПНІ МІСЦЯ" << endl;
    cout <<
    "=====
=====\\n";

    for (const auto& current_flight : flightList) {
        cout << setw(10) << current_flight.getRaceNo()
            << setw(25) << current_flight.getRouteString()
            << setw(25) << fromListToString(current_flight.getStops())
            << setw(20) << current_flight.getDepartureTime()
            << setw(20) << current_flight.getFlightTime()
            << setw(20) << (current_flight.isFull() ? "FULL" :
to_string(current_flight.getAvailableSeats()))
            << endl;
    }

    cout <<
    "=====
=====\\n\\n";
}

Flight* FlightManager::findNearsetFlight(string destination) {
    for (auto& current_flight : flights) {
        if (toLower(current_flight.getRoute().second) == toLower(destination)
            && current_flight.getAvailableSeats() > 0)
        {
            return &current_flight;
        }
    }
    cout << "[ПОМИЛКА] Немає жодного доступного рейсу.\n" << endl;
    return nullptr;
}

void FlightManager::bookTickets(User& user) {
    cout << "Введіть пункт призначення: ";
    string destination;
    getline(cin, destination);

    Flight* flight = findNearsetFlight(destination);
    if (!flight) return;

    cout << "Найближчий рейс до вказаного місця знайдено: " << endl;
    list<Flight> temp = { *flight };
    showFlightList(temp);
}

```

```

    if (flight->isFull()) {
        cout << "[ПОМИЛКА] Всі місця даного рейсу зайняті.\n" << endl;
        return;
    }

    cout << "Будь ласка, введіть кількість квитків, яку Ви хотіли б придбати: ";
    int amount = safeInput();

    if (amount <= 0) {
        cout << "[ПОМИЛКА] Неправильно вказана кількість квитків(<=0).\n" << endl;
        return;
    }
    if (amount > flight->getAvailableSeats()) {
        cout << "[ПОМИЛКА] Недостатньо місць для бронювання.\n" << endl;
        return;
    }

    cout << "Будь ласка, підтвердьте Вашу покупку (y/n): ";
    string confirm;
    getline(cin, confirm);
    if (confirm != "y" && confirm != "Y") {
        cout << "[ІНФО] Покупку було скасовано.\n" << endl;
        return;
    }

    flight->setAvailableSeats(flight->getAvailableSeats() - amount);
    cout << "[ІНФО] Дякуємо за Вашу покупку!\n" << endl;
    generateBoardingPass(user, *flight, amount);
}

void FlightManager::generateBoardingPass(User& user, Flight& flight, int ticketCount) {
    ofstream file("boarding_passes.csv", ios::app);
    if (!file.is_open()) {
        cout << "[ПОМИЛКА] Не вдалося відкрити файл boarding_passes.csv\n" << endl;
        return;
    }

    file << user.getLog() << ", "
        << flight.getRaceNo() << ", "
        << flight.getRoute().first << "-" << flight.getRoute().second << ", "
        << flight.getDepartureTime() << ", "
        << ticketCount << "\n";

    file.close();
    cout << "[ІНФО] Посадковий квиток(відомість) успішно створено!\n" << endl;
}

void FlightManager::add(){
    int flightNo;
    string route;
    string stops;
    string departureTime;
    int flightTime;
    int availableSeats;

    cout << "Введіть номер рейсу: "; flightNo = safeInput();
    cout << "Введіть маршрут для рейсу: "; getline(cin, route);
    cout << "Введіть зупинки для рейсу: "; getline(cin, stops); if (stops.empty()) stops = "none";
    cout << "Введіть час відправлення для рейсу: "; getline(cin, departureTime);
    cout << "Введіть тривалість польоту для рейсу(в год): "; flightTime = safeInput();
    cout << "Введіть кількість вільних місць в рейсі: "; availableSeats = safeInput();

    if (isValidNo(flightNo) && isValidRoute(route) && isValidTimeFormat(departureTime)) {
        flights.push_back(Flight(flightNo, fromStringToPair(route), fromStringToList(stops),
            departureTime, flightTime, availableSeats));
        cout << "[ІНФО] Рейс було успішно створено!\n" << endl;
    }
    else cout << "[ПОМИЛКА] Невдалося створити новий рейс через вище перераховані проблеми.\n" <<
endl;
}

void FlightManager::remove() {
    cout << "Введіть номер рейсу, який Ви хотіли б видалити: ";
    int value = safeInput();
    for (auto it = flights.begin(); it != flights.end(); ) {

```

```

        if (it->getRaceNo() == value) {
            it = flights.erase(it);
            cout << "[ІНФО] Рейс успішно видалено!\n" << endl;
            return;
        }
        else it++;
    }
    cout << "[ПОМИЛКА] Не існує рейсів із таким номером.\n" << endl;
}

void FlightManager::edit() {
    cout << "Введіть номер рейсу, який Ви хотіли б відредагувати: ";
    int flightNo = safeInput();
    auto it = find_if(flights.begin(), flights.end(),
        [flightNo](Flight& f) { return f.getRaceNo() == flightNo; });

    if (it == flights.end()) {
        cout << "[ПОМИЛКА] Не існує рейсів із таким номером.\n" << endl;
        return;
    }
    Flight& flightToEdit = *it;

    string newStringValue;
    int newIntValue;

    cout << "Що саме Ви б хотіли змінити:\n";
    cout << "1. Номер рейсу.\n"
        << "2. Маршрут рейсу.\n"
        << "3. Зупинки рейсу.\n"
        << "4. Час відправки рейсу.\n"
        << "5. Тривалість рейсу.\n"
        << "6. Кількість вільних місць рейсу.\n"
        << "0. Повернутися до головного меню.\n";
    cout << "Ваш вибір: ";
    int option = safeInput();
    switch (option) {
        case 0:
            cout << "Повернення до головного меню...\n" << endl;
            return;
        case 1:
            cout << "Введіть новий номер рейсу: ";
            newIntValue = safeInput();
            if (!isValidNo(newIntValue)) return;
            flightToEdit.setNo(newIntValue);
            cout << "[ІНФО] Успішно змінено!\n" << endl;
            break;
        case 2:
            cout << "Введіть новий маршрут рейсу: "; getline(cin, newStringValue);
            if (!isValidRoute(newStringValue)) return;
            flightToEdit.setRoute(fromStringToPair(newStringValue));
            cout << "[ІНФО] Успішно змінено!\n" << endl;
            break;
        case 3:
            cout << "Введіть рейсові зупинки: ";
            getline(cin, newStringValue);
            if (newStringValue.empty()) newStringValue = "none";
            flightToEdit.setStops(fromStringToList(newStringValue));
            cout << "[ІНФО] Успішно змінено! Поточні зупинки тепер: "
                << flightToEdit.getTotalStops() << endl;
            break;
        case 4:
            cout << "Введіть новий час відправки рейсу: "; getline(cin, newStringValue);
            if (!isValidTimeFormat(newStringValue)) return;
            flightToEdit.setDepartureTime(newStringValue);
            cout << "[ІНФО] Успішно змінено!\n" << endl;
            break;
        case 5:
            cout << "Введіть нову тривалість польоту рейсу: "; newIntValue = safeInput();
            flightToEdit.setFlightTime(newIntValue);
            cout << "[ІНФО] Успішно змінено!\n" << endl;
            break;
        case 6:
            cout << "Введіть нове значення кількості вільних місць в рейсі: "; newIntValue =
safeInput();
            flightToEdit.setAvailableSeats(newIntValue);
            cout << "[ІНФО] Успішно змінено!\n" << endl;
            break;
    }
}

```

```

        default:
            cout << "[ПОМИЛКА] Невірна опція!\n" << endl;
            break;
    }
}

void FlightManager::search() {
    string keyStringValue;
    int keyIntValue = -1;

    cout << "ПОШУК за:\n"
        << "1. Точною відправки.\n"
        << "2. Точною прибуття.\n"
        << "3. Часом відправки.\n"
        << "4. Номером рейсу.\n"
        << "5. Зупинками.\n"
        << "0. Повернутись до головного меню.\n";
    cout << "Ваш вибір: ";
    int choice = safeInput();

    switch (choice) {
    case 0:
        cout << "Повернення до головного меню...\n" << endl;
        return;
    case 1:
        cout << "Введіть точку відправки: "; getline(cin, keyStringValue);
        showFlightList(findFlights(keyStringValue));
        break;
    case 2:
        cout << "Введіть точку прибуття: "; getline(cin, keyStringValue);
        showFlightList(findFlights("", keyStringValue));
        break;
    case 3:
        cout << "Введіть час відправки: "; getline(cin, keyStringValue);
        if (!isValidTimeFormat(keyStringValue)) return;
        showFlightList(findFlights("", "", keyStringValue));
        break;
    case 4:
        cout << "Введіть номер рейсу: "; keyIntValue = safeInput();
        showFlightList(findFlights("", "", "", keyIntValue));
        break;
    case 5: {
        cout << "Введіть точку проміжної зупинки, яку Ви шукаєте: "; getline(cin,
keyStringValue);
        showFlightList(findFlights("", "", "", keyStringValue));
        break;
    }
    default:
        cout << "[ПОМИЛКА] Невірна опція.\n" << endl;
        return;
    }
}

void FlightManager::searchMultiple() {
    string departure, destination, departureTime, stop;
    string flightNoStr;
    int flightNo = -1;

    cout << "Введіть точку відправки(або залиште поле пустим): "; getline(cin, departure);
    cout << "Введіть точку прибуття(або залишт поле пустим): "; getline(cin, destination);
    cout << "Введіть час відправки(або залиште поле пустим): "; getline(cin, departureTime);
    cout << "Введіть пункти проміжної зупинки(або залиште поле пустим): "; getline(cin, stop);
    cout << "Введіть номер рейсу(або залиште поле пустим): "; getline(cin, flightNoStr);

    if (!flightNoStr.empty()) {
        try {
            flightNo = stoi(flightNoStr);
        }
        catch (...) {
            cout << "[ПОМИЛКА] Номер рейсу має бути числом.\n" << endl;
            return;
        }
    }

    if (!departureTime.empty() && !isValidTimeFormat(departureTime)) return;

    auto results = findFlights(departure, destination, departureTime, stop, flightNo);
    showFlightList(results);
}

```



```

}

void FlightManager::filter() {
    cout << "Фільтрувати рейси за (залиште поля пустими, щоб пропустити):\n";
    string dep, dest, time, stop, numStr;
    int no = -1;

    cout << "Точною відправки: "; getline(cin, dep);
    cout << "Точною прибуття: "; getline(cin, dest);
    cout << "Часом відправки: "; getline(cin, time);
    cout << "Пунктом проміжної зупинки: "; getline(cin, stop);
    cout << "Номером рейсу: "; getline(cin, numStr);

    if (!numStr.empty()) {
        try {
            no = stoi(numStr);
        }
        catch (...) {
            cout << "[ПОМИЛКА] Номер рейсу повинен бути цілним числом.\n" << endl;
            return;
        }
    }

    auto filtered = findFlights(dep, dest, time, stop, no);
    showFlightList(filtered);
}

void FlightManager::sortData() {
    cout << "Сортувати за:" << endl;
    cout << "1. Зростанням.\n"
        << "2. Спаданням.\n" << endl;
    int choice = safeInput();
    if (choice < 1 || choice > 2) {
        cout << "[ПОПЕРЕДЖЕННЯ] Невідомий параметр сортування.\n" << endl;
        return;
    }
    cout << "Сортувати за:" << endl;
    cout << "1. Номером рейсу.\n"
        << "2. Точною відправки.\n"
        << "3. Точною прибуття.\n"
        << "4. Часом відправки.\n"
        << "5. Тривалістю польоту.\n"
        << "6. Кількістю вільних місць.\n";
    int filterKey = safeInput();
    if (filterKey > 6 && filterKey < 1) {
        cout << "[ПОПЕРЕДЖЕННЯ] Невідомий ключ сортування.\n" << endl;
        return;
    }
    list<Flight> sortedFlights;
    if (choice == 1) sortedFlights = sortFlights(flights, filterKey);
    else sortedFlights = sortFlights(flights, filterKey, false);
    showFlightList(sortedFlights);
}

/// FILE FUNCTIONS ///
void FlightManager::loadFromFile() {
    if (!fileExists()) {
        cout << "[ПОПЕРЕДЖЕННЯ]: Файлу '" << filepath << "' не існує. Створення нового
файлу...\n";
        ofstream newFile(filepath);
        newFile.close();
        return;
    }

    if (!validateFileFormat("csv")) {
        cout << "[ПОМИЛКА] Некоректний формат файлу. Очікується .csv файл.\n";
        return;
    }

    ifstream file(filepath);
    if (!file.is_open()) {
        cout << "[ПОМИЛКА] Невдалося відкрити файл '" << filepath << "'.\n\n";
        return;
    }

    string line;
    int raceNo;
    pair<string, string> route;

```

```

string stopsRaw, departureTime;
int flightTime, availableSeats;

while (getline(file, line)) {
    if (line.empty()) continue;

    stringstream ss(line);
    string token;

    try {
        getline(ss, token, ','); raceNo = stoi(token);
        getline(ss, token, ','); route = fromStringToPair(token);
        getline(ss, stopsRaw, ',');
        getline(ss, departureTime, ',');
        getline(ss, token, ','); flightTime = stoi(token);
        getline(ss, token, ','); availableSeats = stoi(token);

        list<string> stops = fromStringToList(stopsRaw);
        flights.push_back(Flight(raceNo, route, stops, departureTime, flightTime,
availableSeats));
    }
    catch (const exception& e) {
        cout << "[ПОМИЛКА] під час розподілення: " << e.what() << "\n\n";
    }
}

file.close();
//cout << "[ІНФО] Завантажено " << countOfElements() << " рейсів із файлу.\n";
//showFlightList(flights);
}

void FlightManager::saveToFile() {
    if (!validateFileFormat("csv")) {
        cout << "[ПОМИЛКА] Некоректний формат файлу. Очікується .csv файл для зберігання.\n";
        return;
    }

    ofstream file(filepath);
    if (!file.is_open()) {
        cout << "[ПОМИЛКА] Невдалося відкрити файл '" << filepath << "' для запису.\n";
        return;
    }

    for (const auto& current_flight : flights) {
        file << current_flight.toCSV() << "\n";
    }

    file.close();
    cout << "[ІНФО] Збережено " << flights.size() << " рейсів до '" << filepath << "'.\n\n";
}

/// GETTERS && SETTERS ///
list<Flight> FlightManager::getFlights() {
    return flights;
}

```

Програмний модуль «UserManager»

— Вміст UserManager.h

```

#pragma once
#include "IManagable.h"
#include "User.h"
#include "FileHandler.h"

using namespace std;

/**
 * @class UserManager
 * @brief Клас для керування користувачами системи (адміністраторів і звичайних користувачів).
 * @details
 * Клас 'UserManager' відповідає за всі операції, пов'язані з обліковими записами користувачів:
 * - Додавання, видалення та перегляд користувачів.
 * - Авторизація користувачів (вхід у систему).
 * - Збереження й завантаження даних користувачів у/з файлу.
 * - Вивід довідкових меню для адміністратора та користувача.

```

```

*
* Наслідує інтерфейс @ref IManagable (для основних CRUD-операцій) і клас @ref FileHandler (для
* роботи з файлами).
*/
class UserManager : public IManagable, public FileHandler
{
public:
    /// ===== CONSTRUCTORS & DESTRUCTOR ===== ///
    /**
     * @brief Конструктор за замовчуванням.
     * @details Ініціалізує об'єкт без початкових користувачів.
     */
    UserManager();

    /**
     * @brief Конструктор, який приймає шлях до файлу з користувачами.
     * @param filePath Шлях до файлу (наприклад, "users.txt").
     */
    UserManager(const string& filePath);

    /**
     * @brief Конструктор із початковим списком користувачів.
     * @param userList Список користувачів для ініціалізації менеджера.
     */
    UserManager(list<User> userList);

    /**
     * @brief Деструктор класу.
     * @details Під час знищення об'єкта автоматично виконує збереження даних користувачів у файл.
     */
    ~UserManager();

    /// ===== MAIN FUNCTIONALITY ===== ///
    /**
     * @brief Додає нового користувача до системи.
     * @details Перевіряє унікальність логіна і пароля перед додаванням.
     */
    void add() override;

    /**
     * @brief Видаляє користувача із системи за логіном.
     * @details Неможливо видалити адміністратора.
     */
    void remove() override;

    /**
     * @brief Відображає список усіх зареєстрованих користувачів.
     * @details Показує логін, пароль і роль кожного користувача.
     */
    void showUserList();

    /**
     * @brief Виводить довідкове меню адміністратора.
     * @details Містить опис усіх можливостей адміністратора системи.
     */
    void showAdminHelp();

    /**
     * @brief Виводить довідкове меню звичайного користувача.
     * @details Містить опис усіх дій, доступних користувачу: пошук рейсів, купівля квитків тощо.
     */
    void showUserHelp();

    /// ===== FILE OPERATIONS ===== ///
    /**
     * @brief Завантажує дані користувачів із текстового файлу.
     * @details
     * Формат файлу: `username:password`
     * Якщо у файлі є користувач із ім'ям "admin", йому автоматично присвоюється роль
     адміністратора.
     */
    void loadFromFile();

    /**
     * @brief Зберігає список користувачів до текстового файлу.
     * @details
     * Формат запису: `username:password`
     * Кожен користувач зберігається в окремому рядку.

```

```

    */
    void saveToFile();

    /// ===== IManagable OVERRIDES ===== ///
    /**
     * @brief Метод для редагування даних користувача.
     * @details Поки не реалізований.
     */
    void edit() override;

    /**
     * @brief Метод для сортування списку користувачів.
     * @details Поки не реалізований.
     */
    void sortData() override;

    /**
     * @brief Метод для фільтрації користувачів за певними критеріями.
     * @details Поки не реалізований.
     */
    void filter() override;

    /**
     * @brief Метод для пошуку користувачів.
     * @details Поки не реалізований.
     */
    void search() override;

    /**
     * @brief Пошук користувача за іменем.
     * @param username Ім'я користувача.
     * @return Вказівник на знайденого користувача або nullptr, якщо не знайдено.
     */
    User* find(const string& username);

    /// ===== AUTHENTICATION ===== ///
    /**
     * @brief Перевіряє правильність логіна та пароля користувача.
     * @param username Ім'я користувача.
     * @param password Пароль користувача.
     * @param roleOut Повертає роль користувача ("admin" або "user"), якщо вхід успішний.
     * @return true, якщо авторизація успішна, false – у разі помилки.
     */
    bool login(const string& username, const string& password, string& roleOut) const;

private:
    list<User> userList;
};

```

– Bmict UserManager.cpp

```

#include "UserManager.h"

using namespace std;

/// CONSTRUCTORS & DESTRUCTOR ///
UserManager::UserManager() {}
UserManager::UserManager(const string& filePath) : FileHandler(filePath) {}
UserManager::UserManager(list<User> userList) : userList(userList) {}
UserManager::~UserManager() {
    saveToFile();
    cout << "[ІНФО] Дані користувачів успішно збережені до users.txt" << endl;
}

/// MAIN METHODS ///
void UserManager::add() {
    string username, password;
    bool canBeAdded = true;
    cout << "Введіть ім'я нового користувача: ";
    getline(cin, username);
    cout << "Введіть пароль нового користувача: ";
    getline(cin, password);
    for (auto current_user : userList) {
        if (current_user.getLog() == username || current_user.getPass() == password) {
            canBeAdded = false;
        }
    }
}

```

```

    if (canBeAdded == true) {
        userList.push_back(User(username, password, "user"));
        cout << "[ІНФО] Користувача було успішно додано!\n" << endl;
    }
    else cout << "[ПОМИЛКА]: Не вдалося додати нового користувача\
        - ім'я користувача або такий пароль уже існує.\n" << endl;
}

void UserManager::remove() {
    string username;
    cout << "Введіть ім'я користувача: ";
    getline(cin, username);
    if (username == "admin") {
        cout << "[ПОМИЛКА] людино, ну, невже Ви хочете видалити себе із цього світу?\n" <<
endl;
        return;
    }
    for (auto it = userList.begin(); it != userList.end(); ) {
        if (it->getLog() == username) {
            it = userList.erase(it);
            cout << "[ІНФО] Користувача було успішно видалено!\n" << endl;
            return;
        }
        else it++;
    }
    cout << "[ПОМИЛКА] В базі немає такого користувача.\n" << endl;
}

void UserManager::showUserList() {
    cout << left;
    cout << "=====\n";
    cout << setw(20) << "ІМ'Я КОРИСТУВАЧА"
    << setw(20) << "ПАРОЛЬ"
    << setw(10) << "РОЛЬ" << endl;
    cout << "=====\n";

    for (auto i : userList) {
        cout << setw(20) << i.getLog()
        << setw(20) << i.getPass()
        << setw(10) << i.getRole()
        << endl;
    }

    cout << "=====\n\n";
}

/// ADMIN HELP ///
void UserManager::showAdminHelp() {
    cout << "\n===== МЕНЮ ДОВІДКИ АДМІНІСТРАТОРА =====\n";
    cout << "Як адміністратор, ви маєте повний доступ до керування користувачами та рейсами.\n\n";

    cout << "->>>КЕРУВАННЯ КОРИСТУВАЧАМИ<<-\n";
    cout << " (1) Додати нового користувача - створює новий обліковий запис із логіном і
паролем.\n";
    cout << " (2) Видалити користувача - видаляє існуючого користувача (крім адміністратора).\n";
    cout << " (3) Показати список користувачів - відображає всіх зареєстрованих користувачів та
їх ролі.\n";
    cout << " (4) Змінити пароль - змінює ваш власний пароль адміністратора.\n";
    cout << " (5) Змінити ім'я користувача - змінює ваш логін адміністратора.\n";
    cout << " (6) Показати інформацію профілю - відображає ваш поточний профіль
адміністратора.\n\n";

    cout << "->>>КЕРУВАННЯ РЕЙСАМИ<<-\n";
    cout << " (7) Додати новий рейс - створює новий рейс із маршрутом, зупинками та кількістю
місць.\n";
    cout << " (8) Редагувати рейс - змінює дані рейсу (маршрут, час, місця тощо).\n";
    cout << " (9) Видалити рейс - видаляє конкретний рейс за його номером.\n";
    cout << " (10) Показати список рейсів - відображає всі наявні рейси.\n";
    cout << " (11) Пошук - знаходить рейси за одним критерієм (наприклад, пунктом призначення або
номером).\n";
    cout << " (12) Багатокритеріальний пошук - поєднує кілька критеріїв для детального пошуку.\n";
    cout << " (13) Сортувати дані - сортує рейси за тривалістю, часом вильоту або кількістю
місць.\n";
    cout << " (14) Фільтрувати дані - показує лише рейси, які відповідають заданим умовам.\n\n";

    cout << "->>>ІНШЕ<<-\n";
    cout << " (15) Показати довідку (це меню).\n";
    cout << " (0) Вихід до головного меню.\n";
}

```

```

        cout << "=====\\n\\n";
    }

    /// USER HELP ///
    void UserManager::showUserHelp() {
        cout << "\\n===== МЕНЮ ДОВІДКИ КОРИСТУВАЧА =====\\n";
        cout << "Як користувач, ви можете переглядати рейси, купувати квитки та керувати своїм профілем.\\n\\n";

        cout << "->>>ОПЕРАЦІЇ З РЕЙСАМИ<<<-:\\n";
        cout << " (1) Показати рейси – відображає всі доступні рейси.\\n";
        cout << " (2) Пошук – знаходить рейси за одним критерієм (наприклад, пунктом призначення чи часом).\\n";
        cout << " (3) Багатокритеріальний пошук – дозволяє здійснити пошук за кількома параметрами (маршрут, зупинка, час тощо).\\n";
        cout << " (4) Сортувати – впорядковує рейси за часом, тривалістю або кількістю місць.\\n";
        cout << " (5) Фільтрувати – показує лише ті рейси, які відповідають заданим параметрам.\\n";
        cout << " (6) Придбати квиток – дає змогу купити квиток на обраний рейс.\\n\\n";

        cout << "->>>ОПЕРАЦІЇ З ПРОФІЛЕМ<<<-:\\n";
        cout << " (7) Змінити ім'я користувача – оновлює ваш логін облікового запису.\\n";
        cout << " (8) Змінити пароль – оновлює пароль вашого облікового запису.\\n";
        cout << " (9) Показати інформацію профілю – відображає поточні дані вашого профілю.\\n";
        cout << " (10) Переглянути історію посадкових талонів – показує список усіх ваших придбаних квитків.\\n\\n";

        cout << "->>>ІНШЕ<<<-:\\n";
        cout << " (11) Показати довідку (це меню).\\n";
        cout << " (0) Вихід із облікового запису.\\n";
        cout << "=====\\n\\n";
    }

    void UserManager::edit() {
    }
    void UserManager::sortData() {
    }
    void UserManager::filter() {
    }
    void UserManager::search() {
    }

    User* UserManager::find(const string& username) {
        for (auto& user : userList) {
            if (user.getLog() == username) {
                return &user;
            }
        }
        return nullptr;
    }

    /// FILE METHODS ///
    void UserManager::loadFromFile() {
        if (!fileExists()) {
            cout << "[ПОПЕРЕДЖЕННЯ] Файлу '" << filepath << "' не існує. Створення нового файлу...\\n";
            ofstream newFile(filepath);
            newFile.close();
            return;
        }

        if (!validateFileFormat("txt")) {
            cout << "[ПОМИЛКА] Невірний формат файлу. Очікується .txt файл.\\n";
            return;
        }

        ifstream file(filepath);
        if (!file.is_open()) {
            cout << "[ПОМИЛКА] Не вдалося відкрити файл - '" << filepath << endl << endl;
            return;
        }
    }

```

```

}

string data;
string username, password, role;

while (getline(file, data)) {
    if (data.empty())
        continue;

    size_t firstColon = data.find(':');
    if (firstColon == string::npos)
        continue;

    size_t secondColon = data.find(':', firstColon + 1);

    if (secondColon == string::npos) {
        username = data.substr(0, firstColon);
        password = data.substr(firstColon + 1);

        role = (username == "admin") ? "admin" : "user";
    }
    else {
        username = data.substr(0, firstColon);
        password = data.substr(firstColon + 1, secondColon - firstColon - 1);
        role = data.substr(secondColon + 1);

        if (role != "admin" && role != "user") {
            role = (username == "admin") ? "admin" : "user";
        }
    }

    userList.push_back(User(username, password, role));
}

file.close();
//cout << "[ІНФ0] Завантажено " << countOfElements() << " користувачів із файлу.\n";
}

void UserManager::saveToFile() {
    if (!validateFileFormat("txt")) {
        cout << "[ПОМИЛКА] Невірний формат файлу для зберігання. Очікується .txt файл.\n";
        return;
    }

    ofstream file(filepath);
    if (!file.is_open()) {
        cout << "[ПОМИЛКА] Не вдалося відкрити файл - " << filepath << endl << endl;
        return;
    }

    for (auto& current_user : userList) {
        file << current_user.getLog()
            << ":" << current_user.getPass()
            << ":" << current_user.getRole()
            << endl;
    }

    file.close();
    cout << "[ІНФ0] Збережено " << userList.size() << " користувачів до файлу.\n";
}

bool UserManager::login(const string& username, const string& password, string& roleOut) const {
    for (auto user : userList) {
        if (user.getLog() == username && user.getPass() == password) {
            roleOut = user.getRole();
            return true;
        }
    }
    return false;
}

```

Програмний модуль «Flight»

– Вміст Flight.h

#pragma once

```

#include <string>
#include <list>
#include <utility>

using namespace std;

/**
 * @class Flight
 * @brief Клас, що представляє інформацію про авіарейс.
 * @details
 * Клас 'Flight' зберігає основні дані про рейс:
 * - номер рейсу;
 * - маршрут (пункт відправлення та призначення);
 * - список проміжних зупинок;
 * - час відправлення;
 * - тривалість польоту;
 * - кількість доступних місць.
 *
 * Також надає допоміжні методи для:
 * - конвертації рейсу у CSV-формат;
 * - перевірки наявності зупинок;
 * - підрахунку їх кількості;
 * - визначення, чи заповнений рейс.
 */
class Flight
{
public:
    /// ===== CONSTRUCTORS & DESTRUCTOR ===== ///
    /**
     * @brief Конструктор за замовчуванням.
     * @details Створює "порожній" рейс з базовими значеннями:
     * - номер = 0;
     * - маршрут = "none"->"none";
     * - зупинки = {"none"};
     * - час = "00:00";
     * - тривалість = 0;
     * - місць = 0.
     */
    Flight();

    /**
     * @brief Конструктор з параметрами.
     * @param _raceNo Номер рейсу.
     * @param _route Пара міст (відправлення, призначення).
     * @param _stops Список проміжних зупинок.
     * @param _departureTime Час відправлення у форматі "ГГ:ХХ".
     * @param _flightTime Тривалість польоту в хвиликах.
     * @param _availableSeats Кількість вільних місць.
     */
    Flight(const int& _raceNo,
           const pair<string, string>& _route,
           const list<string>& _stops,
           const string& _departureTime,
           const int& _flightTime,
           const int& _availableSeats);

    /**
     * @brief Копіюючий конструктор.
     * @param other Інший об'єкт Flight для копіювання.
     */
    Flight(const Flight& other);

    /**
     * @brief Конструктор переміщення.
     * @param other Інший об'єкт Flight для переміщення.
     */
    Flight(Flight&& other) noexcept;

    /**
     * @brief Деструктор.
     * @details Виконує очищення пам'яті (автоматично).
     */
    ~Flight();

    /// ===== GETTERS ===== ///
    /**
     * @brief Отримує номер рейсу.
     * @return Ціле число - номер рейсу.
     */

```



```

    */
    int getRaceNo() const;

    /**
     * @brief Отримує маршрут рейсу.
     * @return Пара міст (відправлення, призначення).
     */
    const pair<string, string>& getRoute() const;

    /**
     * @brief Отримує список зупинок.
     * @return Список назв зупинок.
     */
    const list<string>& getStops() const;

    /**
     * @brief Отримує час відправлення.
     * @return Рядок у форматі "ГГ:ХХ".
     */
    const string& getDepartureTime() const;

    /**
     * @brief Отримує тривалість польоту.
     * @return Тривалість у хвиликах.
     */
    int getFlightTime() const;

    /**
     * @brief Отримує кількість доступних місць.
     * @return Кількість місць.
     */
    int getAvailableSeats() const;

    /// ===== SETTERS ===== ///
    /**
     * @brief Встановлює новий номер рейсу.
     * @param newNo Нове значення номера рейсу.
     */
    void setNo(int newNo);

    /**
     * @brief Встановлює новий маршрут.
     * @param newRoute Пара міст (відправлення, призначення).
     */
    void setRoute(pair<string, string> newRoute);

    /**
     * @brief Встановлює новий список зупинок.
     * @param newStops Новий список назв зупинок.
     */
    void setStops(list<string> newStops);

    /**
     * @brief Встановлює новий час відправлення.
     * @param newDepartureTime Новий час у форматі "ГГ:ХХ".
     */
    void setDepartureTime(string newDepartureTime);

    /**
     * @brief Встановлює нову тривалість польоту.
     * @param newFlightTime Тривалість у хвиликах.
     */
    void setFlightTime(int newFlightTime);

    /**
     * @brief Встановлює кількість вільних місць.
     * @param newAvailableSeats Нове значення кількості місць.
     */
    void setAvailableSeats(int newAvailableSeats);

    /// ===== MAIN FUNCTIONALITY ===== ///
    /**
     * @brief Перевіряє, чи має рейс певну зупинку.
     * @param stopName Назва зупинки для перевірки.
     * @return true, якщо така зупинка існує; false – якщо ні.
     */
    bool hasStop(const string& stopName) const;

```

```

/**
 * @brief Отримує маршрут у вигляді рядка.
 * @return Рядок формату "Відправлення - Призначення".
 */
string getRouteString() const;

/**
 * @brief Обчислює загальну кількість зупинок.
 * @return Кількість зупинок, або 0, якщо їх немає.
 */
int getTotalStops() const;

/**
 * @brief Перевіряє, чи заповнено всі місця у рейсі.
 * @return true, якщо вільних місць немає.
 */
bool isFull() const;

/**
 * @brief Перетворює рейс у рядок CSV-формату.
 * @details Формат:
 * `<номер>,<маршрут>,<зупинки через ;>,<час>,<тривалість>,<місця>`
 * @return Рядок з даними про рейс.
 */
string toCSV() const;

private:
    int raceNo;
    pair<string, string> route;
    list<string> stops;
    string departureTime;
    int flightTime;
    int availableSeats;
};

```

— Bmict Flight.cpp

```

#include "Flight.h"
#include <string>
#include <sstream>

/// CONSTRUCTORS && DESTRUCTOR ///
Flight::Flight()
    : raceNo(0),
    route(make_pair("none", "none")),
    stops({ "none" }),
    departureTime("00:00"),
    flightTime(0),
    availableSeats(0) {
}

Flight::Flight(const int& _raceNo, const pair<string, string>& _route,
    const list<string>& _stops, const string& _departureTime,
    const int& _flightTime, const int& _availableSeats) : raceNo(_raceNo),
    route(_route), stops(_stops), departureTime(_departureTime),
    flightTime(_flightTime), availableSeats(_availableSeats) {
}

Flight::Flight(const Flight& other)
    : raceNo(other.raceNo),
    route(other.route),
    stops(other.stops),
    departureTime(other.departureTime),
    flightTime(other.flightTime),
    availableSeats(other.availableSeats) {
}

Flight::Flight(Flight&& other) noexcept
    : raceNo(move(other.raceNo)),
    route(move(other.route)),
    stops(move(other.stops)),
    departureTime(move(other.departureTime)),
    flightTime(move(other.flightTime)),
    availableSeats(move(other.availableSeats)) {
}

```

```

}

Flight::~Flight() = default;

/// GETTERS && SETTERS ///
int Flight::getRaceNo() const { return raceNo; }
const pair<string, string>& Flight::getRoute() const { return route; }
const list<string>& Flight::getStops() const { return stops; }
const string& Flight::getDepartureTime() const { return departureTime; }
int Flight::getFlightTime() const { return flightTime; }
int Flight::getAvailableSeats() const { return availableSeats; }

/// SETTERS ///
void Flight::setNo(int newNo) { raceNo = newNo; }
void Flight::setRoute(pair<string, string> newRoute) { route = newRoute; }
void Flight::setStops(list<string> newStops) { stops = newStops; }
void Flight::setDepartureTime(string newDepartureTime) { departureTime = newDepartureTime; }
void Flight::setFlightTime(int newFlightTime) { flightTime = newFlightTime; }
void Flight::setAvailableSeats(int newAvailableSeats) { availableSeats = newAvailableSeats; }

/// MAIN METHODS ///
bool Flight::hasStop(const string& stopName) const {
    for (const auto& stop : stops) {
        if (stop == stopName) return true;
    }
    return false;
}

string Flight::getRouteString() const {
    return route.first + " - " + route.second;
}

int Flight::getTotalStops() const {
    if (stops.size() == 1 && stops.front() == "none") return 0;
    return static_cast<int>(stops.size());
}

bool Flight::isFull() const {
    return availableSeats <= 0;
}

string Flight::toCSV() const {
    stringstream ss;
    ss << raceNo << "," << route.first << "-" << route.second << ",";

    bool first = true;
    for (const auto& stop : stops) {
        if (!first) ss << ";";
        ss << stop;
        first = false;
    }

    ss << "," << departureTime
        << "," << flightTime
        << "," << availableSeats;
    return ss.str();
}

```

Програмний модуль «User»

— Вміст User.h

```

#pragma once
#include <string>

using namespace std;

class FlightManager;
class UserManager;

/**
 * @class User
 * @brief Клас, що представляє окремого користувача системи.
 * @details
 * Клас `User` містить інформацію про логін, пароль і роль користувача.
 */

```

```

* Забезпечує основні можливості для роботи користувача в системі:
* - зміна логіна та пароля;
* - перегляд профілю;
* - покупка квитків;
* - перегляд історії посадкових талонів.
*
* Використовується у поєднанні з класами @ref FlightManager та @ref UserManager.
*/
class User
{
public:
    /// ===== CONSTRUCTORS ===== ///
    /**
     * @brief Конструктор за замовчуванням.
     * @details Створює користувача з ім'ям "guest" та роллю "user".
     */
    User();

    /**
     * @brief Конструктор з логіном і паролем.
     * @param _username Ім'я користувача (логін).
     * @param _password Пароль користувача.
     * @details Роль за замовчуванням – "user".
     */
    User(const string& _username, const string& _password);

    /**
     * @brief Конструктор із вказаною роллю.
     * @param _username Ім'я користувача.
     * @param _password Пароль користувача.
     * @param _role Роль користувача ("admin" або "user").
     */
    User(const string& _username, const string& _password, const string& _role);

    /**
     * @brief Копіюючий конструктор.
     * @param other Інший об'єкт User для копіювання.
     */
    User(const User& other);

    /**
     * @brief Конструктор переміщення.
     * @param other Інший об'єкт User для переміщення.
     */
    User(User&& other) noexcept;

    /**
     * @brief Деструктор користувача.
     * @details Викликається автоматично при видаленні об'єкта користувача.
     */
    ~User();

    /// ===== GETTERS ===== ///
    /**
     * @brief Отримує логін користувача.
     * @return Логін у вигляді рядка.
     */
    string getLog();

    /**
     * @brief Отримує пароль користувача.
     * @return Пароль у вигляді рядка.
     */
    string getPass();

    /**
     * @brief Отримує роль користувача.
     * @return "admin" якщо користувач – адміністратор, або "user" інакше.
     */
    string getRole();

    /// ===== MAIN FUNCTIONALITY ===== ///
    /**
     * @brief Процес купівлі квитка користувачем.
     * @details Викликає функцію бронювання у @ref FlightManager.
     * @param flightManager Посилання на об'єкт менеджера рейсів.
     */
    void buyTicket(FlightManager& flightManager);

```

```

/**
 * @brief Змінює пароль користувача.
 * @details
 * - Перевіряє правильність поточного пароля.
 * - Вимагає введення нового пароля.
 * - Виводить повідомлення про успіх або помилку.
 */
void changePassword();

/**
 * @brief Змінює ім'я користувача.
 * @details
 * - Перевіряє, чи не порожнє введене ім'я.
 * - Перевіряє, чи немає користувача з таким логіном у @ref UserManager.
 * - Оновлює логін у поточному об'єкті.
 * @param userManager Менеджер користувачів, необхідний для перевірки унікальності імені.
 */
void changeUsername(UserManager& userManager);

/**
 * @brief Відображає інформацію профілю користувача.
 * @details Виводить логін і роль у консоль у форматованому вигляді.
 */
void showProfile() const;

/**
 * @brief Відображає історію придбаних квитків.
 * @details
 * Зчитує дані з файлу `boarding_passes.csv` і виводить записи,
 * що належать поточному користувачу.
 * Якщо історії немає – виводить попередження.
 */
void viewBoardingPass() const;

private:
string username;
string password;
string role;
};

```

– Bmict User.cpp

```

#include "User.h"
#include "FlightManager.h"
#include "UserManager.h"
#include <string>
#include <iostream>

using namespace std;

/// CONSTRUCTORS && DESTRUCTORS ///
User::User() : username("guest"), password("guest123"), role("user") {}
User::User(const string& _username, const string& _password) :
    username(_username), password(_password), role("user") {}
User::User(const string& _username, const string& _password, const string& _role) :
    username(_username), password(_password), role(_role) {}
User::User(const User& other) :
    username(other.username), password(other.password), role(other.role) {}
User::User(User&& other) noexcept :
    username(move(other.username)), password(move(other.password)), role(move(other.role)) {}
User::~User() {}

/// GETTERS && SETTERS ///
string User::getLog() {
    return this->username;
}

string User::getPass() {
    return this->password;
}

string User::getRole() {
    return role;
}

/// MAIN METHODS ///

```

```

void User::buyTicket(FlightManager& flightManager) {
    flightManager.bookTickets(*this);
}

void User::changePassword() {
    cout << "Введіть Ваш поточний пароль: ";
    string current; getline(cin, current);
    if (current != password) {
        cout << "[ПОМИЛКА] Невірно введений поточний пароль.\n" << endl;
        return;
    }

    cout << "Введіть Ваш новий пароль: ";
    string newPass; getline(cin, newPass);
    if (newPass.empty()) {
        cout << "[ПОМИЛКА] Пароль не може бути пустим.\n" << endl;
        return;
    }

    password = newPass;
    cout << "[ІНФО] Пароль успішно змінено!\n" << endl;
}

void User::changeUsername(UserManager& userManager) {
    cout << "Введіть Ваш новий логін: ";
    string newName;
    getline(cin, newName);

    if (newName.empty()) {
        cout << "[ПОМИЛКА] Логін не може бути пустим.\n" << endl;
        return;
    }

    if (newName == username) {
        cout << "[ПОПЕРЕДЖЕННЯ] Введений логін співпадає з поточним.\n" << endl;
        return;
    }

    if (userManager.find(newName) != nullptr) {
        cout << "[ПОМИЛКА] Користувач із таким логіном уже існує.\n" << endl;
        return;
    }

    User* inList = userManager.find(username);
    if (inList != nullptr) {
        inList->username = newName;
    }

    username = newName;

    cout << "[ІНФО] Логін успішно змінено!\n" << endl;
}

void User::showProfile() const {
    cout << "===== " << endl;
    cout << left
        << setw(15) << "Логін:" << username << "\n"
        << setw(15) << "Роль:" << role << "\n";
    cout << "===== \n" << endl;
}

void User::viewBoardingPass() const {
    ifstream file("boarding_passes.csv");
    if (!file.is_open()) {
        cout << "[ПОМИЛКА] Не вдалось відкрити boarding_passes.txt.\n" << endl;
        return;
    }

    cout << "==== Історія придбаних квитків - " << username << " ===== \n";
    string line;
    bool found = false;

    while (getline(file, line)) {
        stringstream ss(line);
        string name, flightNo, route, departure, tickets;

```

```

getline(ss, name, ',');
getline(ss, flightNo, ',');
getline(ss, route, ',');
getline(ss, departure, ',');
getline(ss, tickets, ',');

if (name == username) {
    cout << "Номер рейсу: " << flightNo
        << " | Маршрут: " << route
        << " | Час відправки: " << departure
        << " | Квитки: " << tickets << endl;
    found = true;
}

if (!found)
    cout << "[ПОПЕРЕДЖЕННЯ] Ви не маєте історії придбання квитків.\n" << endl;

file.close();
cout << "=====\n" << endl;
}

```

Програмний модуль «FileHandler»

– Вміст FileHandler.h

```

#pragma once
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

/**
 * @class FileHandler
 * @brief Абстрактний базовий клас для роботи з файлами.
 * @details
 * Клас `FileHandler` забезпечує базову функціональність для роботи з файлами:
 * - зчитування та запис даних у файл;
 * - перевірка наявності файлу;
 * - підрахунок кількості рядків;
 * - перевірка формату файлу за розширенням.
 *
 * Є базовим класом для менеджерів даних, таких як:
 * - @ref UserManager (робота з користувачами);
 * - @ref FlightManager (робота з рейсами).
 *
 * Має чисто віртуальні методи `loadFromFile()` і `saveToFile()`,
 * які реалізуються в похідних класах.
 */
class FileHandler
{
public:
    /**
     * ===== CONSTRUCTORS & DESTRUCTOR =====
     */
    /**
     * @brief Конструктор за замовчуванням.
     * @details Ініціалізує порожній шлях до файлу.
     */
    FileHandler();

    /**
     * @brief Конструктор із параметром шляху до файлу.
     * @param filepath Шлях до файлу, з яким працюватиме об'єкт.
     */
    FileHandler(const string& filepath);

    /**
     * @brief Віртуальний деструктор.
     * @details Виводить інформаційне повідомлення про знищення об'єкта менеджера файлів.
     */
    virtual ~FileHandler();

    /**
     * ===== GETTERS & SETTERS =====
     */
    /**
     * @brief Отримує поточний шлях до файлу.
     */

```

```

    * @return Рядок із шляхом до файлу.
    */
    string getPath() const;

    /**
     * @brief Встановлює новий шлях до файлу.
     * @param newPath Новий шлях до файлу.
     */
    void setPath(const string& newPath);

    /// ===== ABSTRACT FILE METHODS ===== ///
    /**
     * @brief Завантажує дані з файлу.
     * @details Має бути перевизначений у похідних класах.
     */
    virtual void loadFromFile() = 0;

    /**
     * @brief Зберігає дані до файлу.
     * @details Має бути перевизначений у похідних класах.
     */
    virtual void saveToFile() = 0;

    /// ===== UTILITY METHODS ===== ///
    /**
     * @brief Перевіряє, чи існує файл.
     * @return true, якщо файл існує; false – якщо ні.
     */
    bool fileExists() const;

    /**
     * @brief Підраховує кількість непорожніх рядків у файлі.
     * @details Може бути використано для визначення кількості записів.
     * @return Кількість рядків у файлі, або -1, якщо файл не вдалося відкрити.
     */
    int countOfElements() const;

    /**
     * @brief Перевіряє, чи відповідає файл очікуваному формату.
     * @param expectedExtension Розширення файлу без крапки (наприклад, "txt" або "csv").
     * @return true, якщо розширення збігається; false – якщо ні.
     */
    bool validateFileFormat(const string& expectedExtension) const;

protected:
    string filepath;
};

```

— Вміст FileHandler.cpp

```

#include "FileHandler.h"
#include <iostream>
#include <fstream>

using namespace std;

FileHandler::FileHandler() : filepath("") {}

FileHandler::FileHandler(const string& path) : filepath(path) {}

FileHandler::~FileHandler() {
    cout << "[ІНФО] Менеджер файлів для '" << filepath << "' знищено." << endl;
}

string FileHandler::getPath() const {
    return filepath;
}

void FileHandler::setPath(const string& newPath) {
    filepath = newPath;
}

bool FileHandler::fileExists() const {
    ifstream file(filepath);
    return file.good();
}

int FileHandler::countOfElements() const {

```



```

ifstream file(filepath);
if (!file.is_open()) {
    cout << "[ПОМИЛКА] Неможливо відкрити файл для підрахування рядків.\n";
    return -1;
}

int count = 0;
string line;
while (getline(file, line)) {
    if (!line.empty()) count++;
}

file.close();
return count;
}

bool FileHandler::validateFileFormat(const string& expectedExtension) const {
    size_t dotPos = filepath.find_last_of('.');
    if (dotPos == string::npos) return false;
    string ext = filepath.substr(dotPos + 1);
    return ext == expectedExtension;
}

```